

## Chapter 3

### METHODOLOGY

#### 3.1. Type of Research

This study employed an applied and developmental research approach. Applied research was used because the study is aimed at solving a real-world software engineering problem. In this study, the problem is the difficulty developers encounter in identifying and prioritizing code smells and refactoring opportunities during software maintenance. Unlike theoretical studies, applied research is focused on applying existing techniques and tools to solve real-world problems.

At the same time, it is also a developmental study in nature, as it requires the design and development of a functional software artifact in the form of an integrated decision support system within the IntelliJ IDEA environment. This system integrates multiple existing code smell detection tools and machine learning techniques to assist developers in making informed refactoring decisions. The development of this system follows a structured process that includes requirements analysis, system design, implementation, and evaluation.

In addition, the study has an experimental design as it incorporates machine learning methods, where Random Forest is employed as a technique for analyzing the metrics and data in ranking the identified code smells based on their probabilities of being prioritized by developers. The performance of the proposed system is assessed using metrics and baselines for ranking accuracy.

Overall, the combination of applied, developmental, and experimental research approaches is appropriate for this study, as it enables the construction of a practical decision-support system while also allowing for empirical evaluation of its effectiveness.

3.2. Research Design

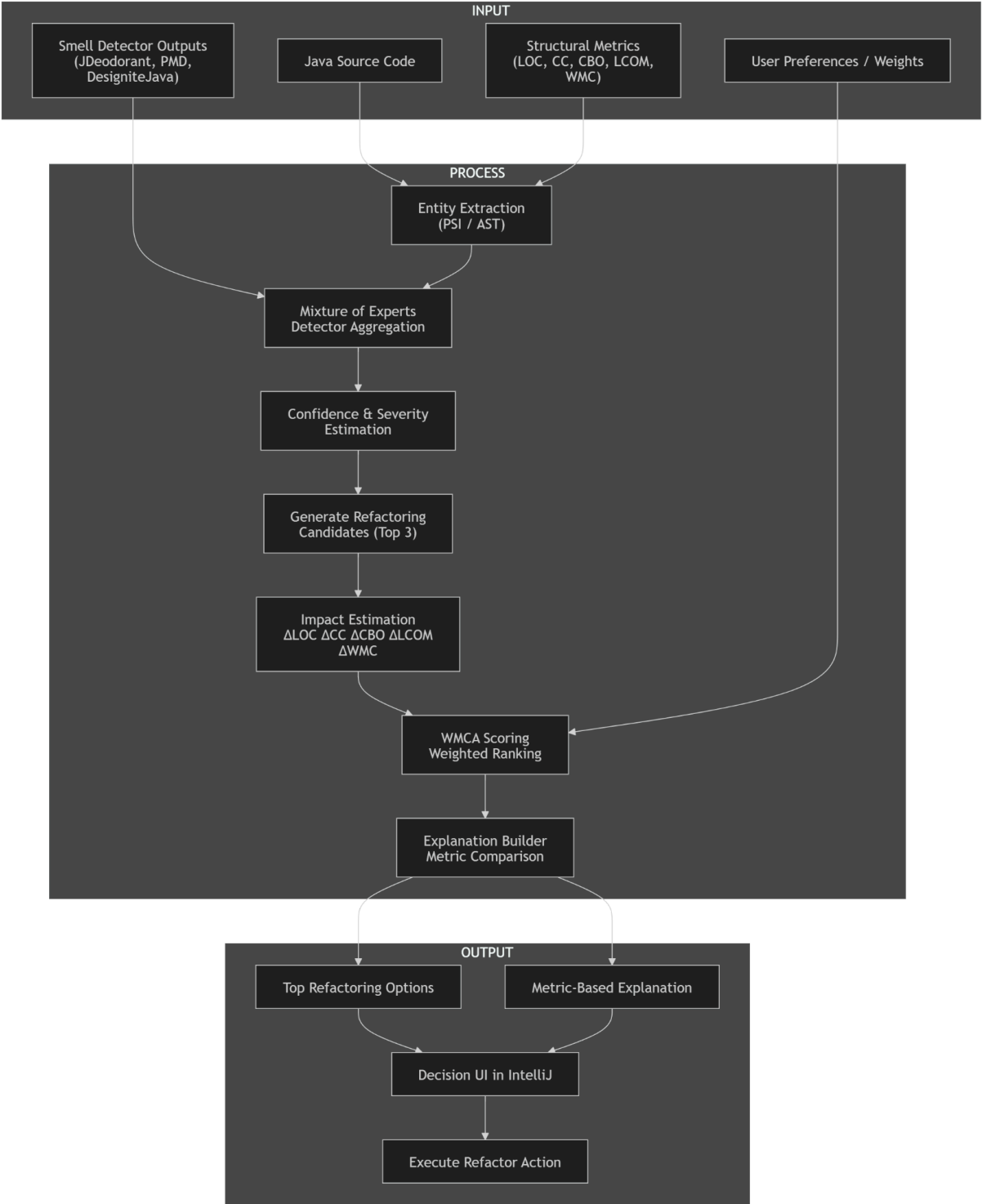


Figure 2. IntelliJ IDEA IPO

This section describes the Input-Process-Output (IPO) conceptual framework used in the study. The framework illustrates how the proposed IntelliJ plugin analyzes Java source code, detects code smells using multiple expert tools, evaluates potential refactoring options, and provides ranked decision-support recommendations.

### **3.2.1. Input Layer**

This section describes the Input-Process-Output (IPO) conceptual framework used in the study. The framework illustrates how the proposed IntelliJ plugin analyzes Java source code, detects code smells using multiple expert tools, evaluates potential refactoring options, and provides ranked decision-support recommendations.

#### **3.2.1.1. Java Source Code**

The primary input of the system is the Java source code obtained from IntelliJ IDEA projects. The system analyzes classes and methods within the project to extract relevant entities for smell detection and refactoring analysis.

#### **3.2.1.2. Code Smell Detector Outputs**

The system integrates outputs from multiple existing smell detection tools which act as experts in the Mixture-of-Experts framework.

The selected tools include:

- JDeodorant
- iPlasma
- DesigniteJava
- JSpIRIT

Each tool analyzes the source code and identifies potential code smells based on their respective detection strategies. The outputs of these tools serve as detection inputs that will be aggregated in the ensemble detection stage.

#### **3.2.1.3. Structural Software Metrics**

The system extracts structural metrics from the Java source code using the IntelliJ PSI/AST structure. These metrics provide quantitative information about code complexity and design quality.

- The extracted metrics include:
- Lines of Code (LOC)
- Cyclomatic Complexity (CC)
- Coupling Between Objects (CBO)
- Lack of Cohesion in Methods (LCOM)
- Weighted Methods per Class (WMC)

These metrics are used to estimate the impact of potential refactoring alternatives.

#### **3.2.1.4. User Preferences and Weights**

Optional user-defined weights can be provided to influence the prioritization of refactoring alternatives. These weights determine the relative importance of different structural metrics during the Weighted Multi-Criteria Analysis (WMCA) stage.

If no user preferences are provided, the system applies default weighting values.

### **3.2.2. Processing Layer**

The processing layer represents the core analysis pipeline of the proposed decision-support system.

#### **3.2.2.1. Entity Extraction**

The system first extracts program entities such as classes and methods using the IntelliJ Program Structure Interface (PSI) and Abstract Syntax Tree (AST). This step prepares the code elements that will be analyzed for potential code smells.

#### **3.2.2.2. Multi-Detector Signal Aggregation**

The outputs of the selected smell detection tools are combined using a Mixture-of-Experts aggregation strategy. This approach integrates the results of multiple detectors to improve the reliability of code smell detection and reduce inconsistencies between tools.

#### **3.2.2.3. Confidence and Severity Estimation**

After aggregating the detector outputs, the system estimates the confidence and severity of each detected code smell. This step determines the likelihood that a detected smell represents a real design problem.

#### **3.2.2.4. Refactoring Candidate Generation**

For each detected code smell, the system generates up to three possible refactoring alternatives. These alternatives represent different ways of resolving the identified smell while preserving the program's functionality.

Examples of generated refactorings may include:

- Extract Method
- Move Method

- Extract Class

These alternatives provide multiple options for the developer rather than enforcing a single automated refactoring.

#### **3.2.2.5. Refactoring Impact Estimation**

Each generated refactoring alternative is evaluated by estimating its expected impact on structural software metrics. The system simulates potential improvements using metric deltas, including:

- $\Delta\text{LOC}$
- $\Delta\text{Cyclomatic Complexity } (\Delta\text{CC})$
- $\Delta\text{CBO}$
- $\Delta\text{LCOM}$
- $\Delta\text{WMC}$

This estimation allows the system to quantitatively compare the effects of different refactoring options.

#### **3.2.2.6. Weighted Multi-Criteria Analysis (WMCA)**

The system applies Weighted Multi-Criteria Analysis to compute a prioritization score for each refactoring alternative.

The scoring function combines the metric improvements and severity scores using predefined or user-defined weights.

This step assigns a prioritization score to each refactoring alternative and ranks the alternatives based on the defined criteria.

#### **3.2.2.7. Explanation Builder**

To support transparent decision-making, the system generates a metric-based explanation for each ranked refactoring alternative. The explanation compares the expected structural metrics before and after refactoring to illustrate the anticipated improvements.

### **3.2.3. Output Layer**

The output layer presents the final decision-support results to the user within the IntelliJ environment.

#### **3.2.3.1. Top Refactoring Options**

The system outputs the Top-3 ranked refactoring alternatives for each detected code smell. These options represent the most promising solutions according to the WMCA prioritization model.

#### **3.2.3.2. Metric-Based Explanation**

Each recommended refactoring alternative includes a detailed explanation showing the expected improvements in software metrics. This allows users to understand the reasoning behind the ranking.

#### **3.2.3.3. Decision Interface in IntelliJ**

The recommendations and explanations are presented through an IntelliJ plugin interface that allows developers to review and compare the suggested refactoring options.

#### **3.2.3.4. Refactoring Execution**

After reviewing the recommendations, the user may choose to apply one of the suggested refactoring actions. The selected refactoring is then executed using IntelliJ's built-in refactoring mechanisms to ensure safe code transformation.

#### **3.2.3.5. Code Smell Summary Report**

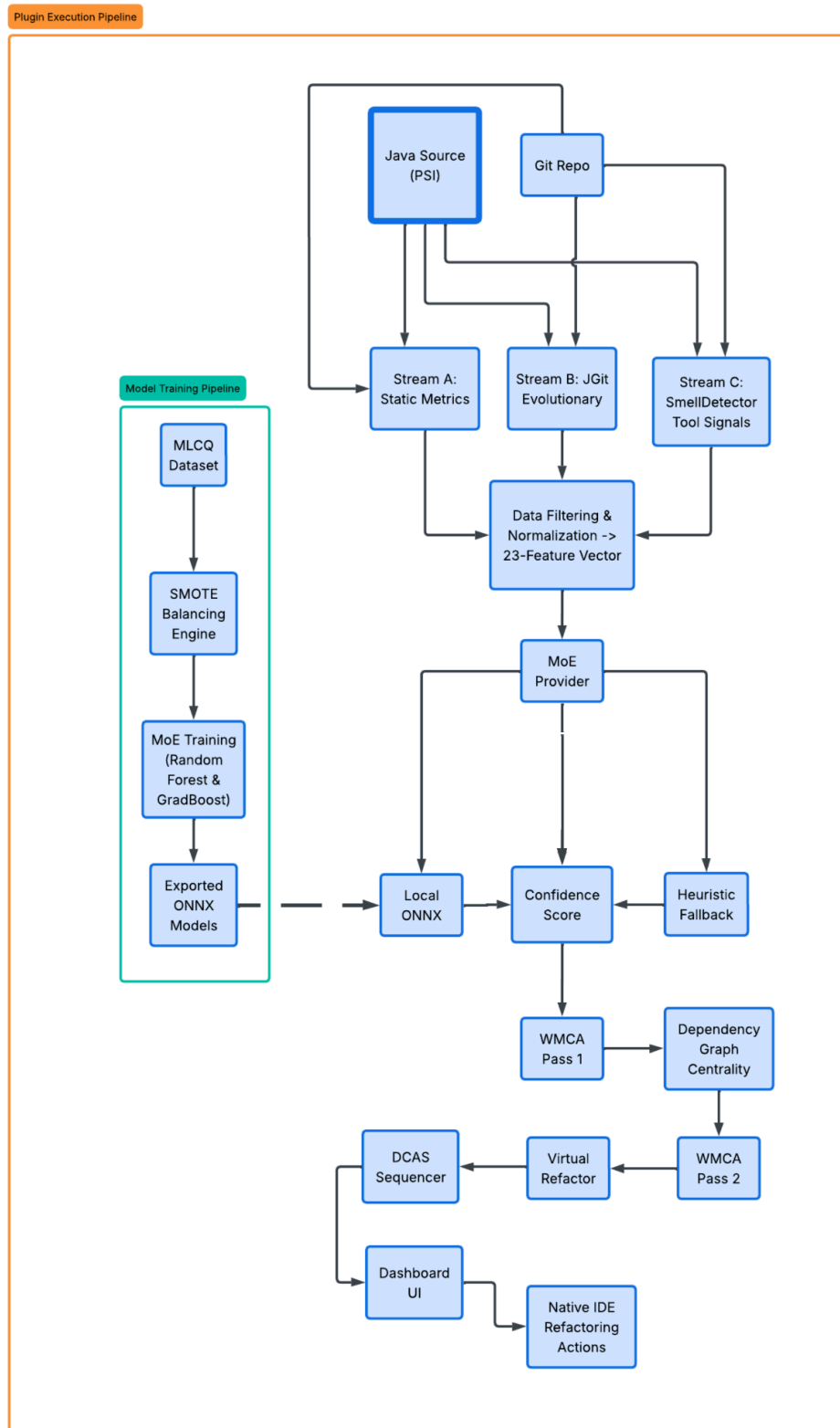
As a further aid in documentation and offline usage a feature is provided that creates a detailed code smell report. Once the project scan is complete a structured summary document can be exported that lists all code smells identified, their precise locations, confidence scores provided by the machine learning component, and prioritized refactoring recommendations generated by the WMCA engine.

### **3.3. System Architecture**

The system architecture is essentially a blueprint that underlies the system workflow, providing a clear representation of how specialized software modules, machine learning algorithms, and user interfaces cohesively integrate with each other in providing build-safe refactoring recommendations. To provide a thorough overview of the cohesive integration, this section maps the system from a functional data pipeline, human-computer interaction, and a chronological perspective in the background processes.



### 3.3.1. Functional Architecture



### Figure 3. Functional System Architecture

Figure 3 illustrates the system architecture as a multi-phase functional pipeline for converting raw software artifacts to refactoring insights. The architecture is divided into two primary interconnected workflows: the Model Training Pipeline and the Plugin Execution Pipeline.

In the Model Training Pipeline, the process begins with the ingestion of raw instances from the MLCQ Dataset. Due to the inherent class imbalance within software metric datasets, the data is passed through a SMOTE Balancing Engine to synthesize minority classes and ensure equitable distribution. The balanced instances are then introduced to the MoE Training module, where base experts are trained using methods like Random Forest and Gradient Boosting. Finally, the Exported ONNX Models package these trained models to serve as the decision-making mechanism for the plugin's local inference engine.

The Plugin Execution Pipeline represents the runtime actions executed within the IntelliJ IDE. The process starts with two raw inputs: the Java Source Code (parsed via IntelliJ's PSI) and the Git Repo (the project's version control history). These inputs feed into three parallel feature extraction streams:

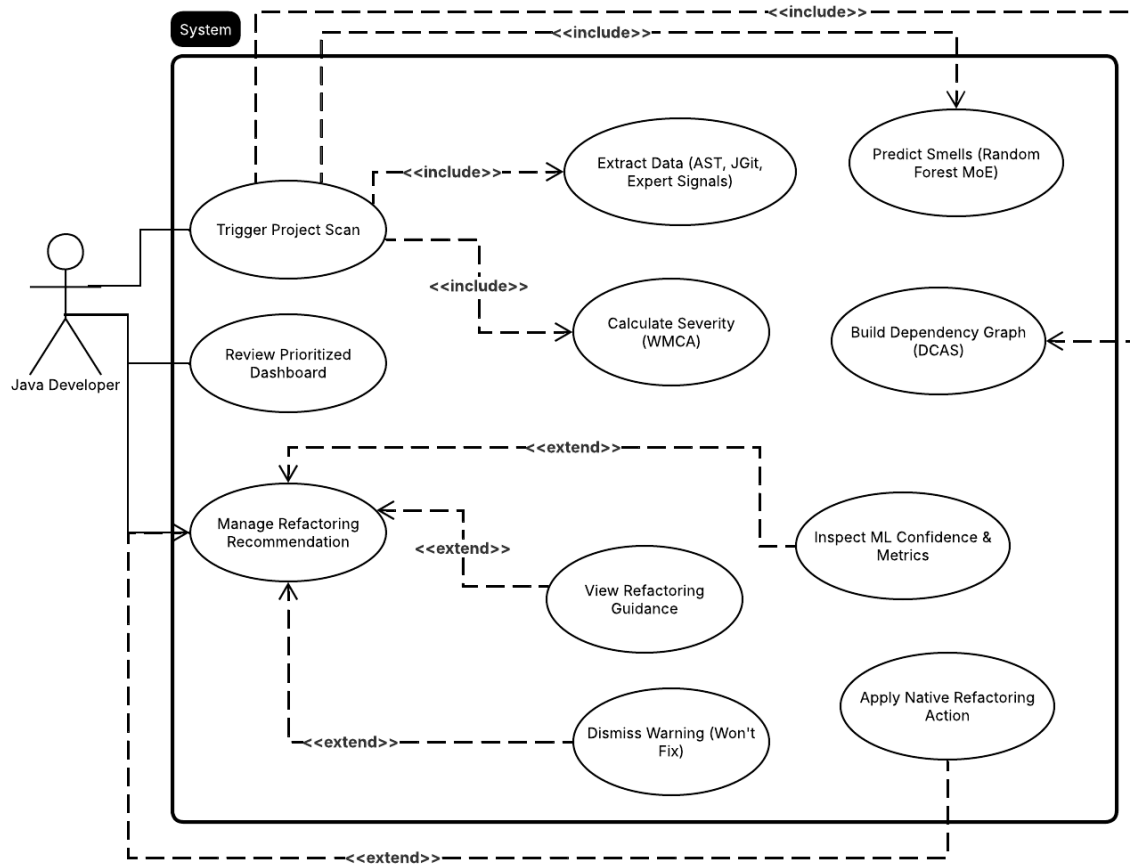
- Stream A (Static Metrics): Extracts structural complexity, size, and security metrics directly from the source code.
- Stream B (JGit Evolutionary): Mines the Git repository to calculate historical metrics like code churn and volatility.
- Stream C (SmellDetector Tool Signals): Simulates peak detection signals from established expert tools using sigmoid-based heuristics.

The outputs from these three streams merge into a Data Filtering and Normalization component, which compiles a standardized 23-Feature Vector. This feature vector is then passed to the MoE Provider, which acts as a routing layer. It directs the data to either the Local ONNX runtime or a Heuristic Fallback, ultimately generating a unified Confidence Score.

Once the Confidence Score is determined, the candidate enters the prioritization and sequencing phase. It first passes through WMCA Pass 1 for initial scoring, then flows into the Dependency Graph Centrality module to evaluate structural importance. This architectural context feeds into WMCA Pass 2 for a final weighted priority calculation. The system then performs a Virtual Refactor simulation to estimate structural improvements before passing the results to the DCAS Sequencer, which safely orders the recommendations to prevent ripple effects.

Finally, the sequenced recommendations are presented to the developer through the Dashboard UI. This interface bridges the gap between analysis and action, allowing users to seamlessly trigger Native IDE Refactoring Actions directly from the prioritized list.

### 3.3.2. UseCase Diagram



**Figure 4. UseCase Diagram**

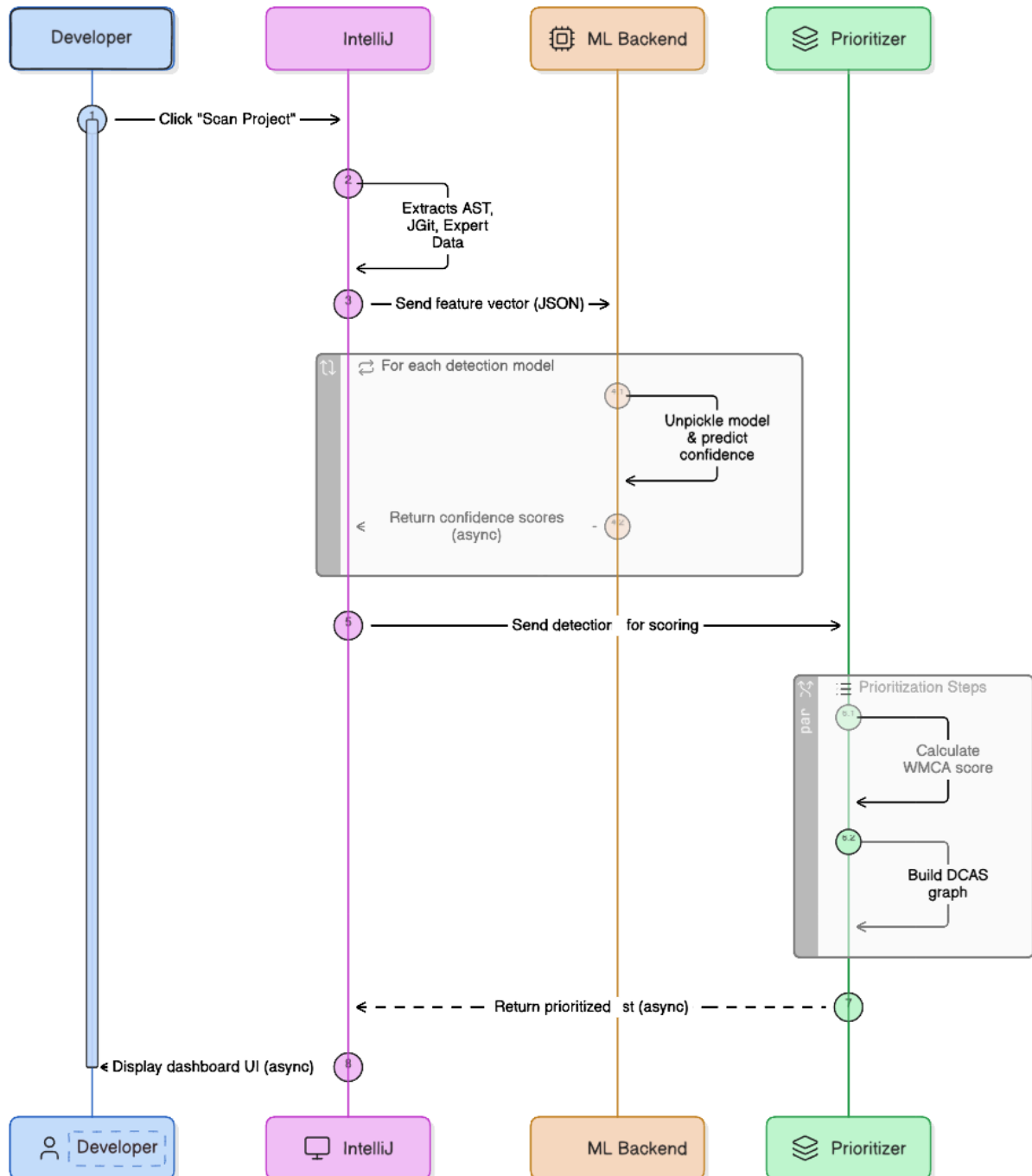
Figure 4 illustrates the Use Case Diagram of the system, which shows the functional boundaries of the system and how the main actors interact with the system boundary. The main actor of this system is the Java Developer who interacts with the plug-in using three main use cases: Trigger Project Scan, Review Prioritized Dashboard, and Manage Refactoring Recommendation.

The diagram utilizes standard UML <<include>> relationships to depict the mandatory background tasks that execute automatically. Once the developer triggers a project scan the

tool automatically captures information related to structure and evolution (AST, JGit, Expert Signals), identifies any code smells based on the Random Forest Mixture-of-Experts (MoE) approach, determines the importance via Weighted Multi-Criteria Analysis (WMCA) and constructs a dependency graph for ensuring safe ordering of code segments (DCAS)

Finally, <<extend>> relationships illustrate the optional interactions available when the developer chooses to manage a specific refactoring recommendation. To ensure the developer retains full authority over the codebase, they can inspect the machine learning confidence scores and metric breakdowns, view plain-English refactoring guidance, dismiss the warning entirely (Won't Fix), or safely transform the code by applying a native IDE refactoring action (such as Extract Class or Move Method). To make sure that the developer is fully in control over the code base, developer is able to check machine learning scores and metrics, get advice in plain English on how to refactor the code, ignore the recommendation altogether (Won't Fix), or safely transform the code by applying a native IDE refactoring action (such as Extract Class or Move Method).

### 3.3.3. Sequence Diagram



### **Figure 5. Sequence Diagram of the System**

Figure 5 presents the Sequence Diagram for the system, detailing the chronological communication and data exchange between four main participants: the Developer, the IntelliJ plugin, the ML Backend, and the Prioritizer. This process starts when the Developer clicks on the "Scan Project" button, thereby initiating IntelliJ to perform a localized AST, JGit, and expert data extraction. Once compiled, IntelliJ transmits the aggregated feature vector, represented in JSON format, to the ML Backend. Following this, ML Backend undergoes an iteration cycle for each of the detection models, wherein it performs un-pickling of the detection models followed by prediction of machine learning confidence scores. In the final step, the ML Backend asynchronously responds back to IntelliJ with these predictions. Subsequently, the detections are passed to the Prioritizer, where a group of prioritization processes is performed sequentially: calculation of WMCA score, followed by construction of DCAS Graph. Lastly, the Prioritizer asynchronously communicates the prioritized build-safe list back to IntelliJ, concluding the process with display of the prioritized dashboard UI to the Developer

### **3.4. Algorithmic Framework**

The proposed framework addresses the "Actionability Gap" in modern refactoring tools by shifting the paradigm from simple detection to prioritized recommendation. The system utilizes a data-driven approach to resolve tool inconsistencies, prioritize smells based on multi-dimensional risks, and sequence them to prevent unintended structural regressions.

#### **3.4.1. Unified Feature Vector**

The framework initiates by transforming raw software artifacts into a standardized numerical representation suitable for machine learning inference. The Unified Feature Vector ( $v$ ) is a

high-dimensional data record that maps heterogeneous signals back to specific Class or Method signatures.

For any code element  $n$  identified by the system, the feature vector  $v_n$  is constructed through the ordered concatenation of three heterogeneous data subsets, which are mapped back to their respective Class or Method signatures

$$v_n = \{M_{structural}, M_{evolutionary}, S_{expert}\}$$

### Equation 1. Unified Feature Vector

$M_{structural}$ : Quantitative characteristics extracted via Recursive AST Traversal using the IntelliJ PSI.

This subset includes:

- **CK Metrics Suite**

Measures object-oriented characteristics including Coupling Between Objects (CBO), Lack of Cohesion in Methods (LCOM), Response for a Class (RFC), and Weighted Methods per Class (WMC)

- **Security-Oriented Indicators**

Class Object Access (COA) and Class Internal Data Access (CIDA)  $M_{evolutionary}$

:Temporal signals mined from the project's version history using JGit.  $S_{expert}$ :

Detection labels synthesized from the rule-based logic of integrated expert toolsets, including JDeodorant, DesigniteJava, iPlasma, and JSpIRIT.

| Index | Field | Type        | Description                                                                  | Block         |
|-------|-------|-------------|------------------------------------------------------------------------------|---------------|
| 0     | wmc   | int         | Weighted Methods per Class — sum of cyclomatic complexity across all methods | CK Structural |
| 1     | cbo   | int         | Coupling Between Objects — distinct external classes referenced              | CK Structural |
| 2     | lcom  | float [0,1] | Lack of Cohesion of Methods (Henderson-Sellers)                              | CK Structural |
| 3     | rfc   | int         | Response for a Class — own methods + externally called methods               | CK Structural |
| 4     | dit   | int         | Depth of Inheritance Tree                                                    | CK Structural |
| 5     | loc   | int         | Lines of Code — non-blank, non-comment source lines at class scope           | CK Structural |
| 6     | ccMax | int         | Maximum Cyclomatic Complexity across all methods (worst single method)       | CK Structural |
| 7     | nom   | int         | Number of                                                                    | Extended      |



|    |               |             |                                                                                |                        |
|----|---------------|-------------|--------------------------------------------------------------------------------|------------------------|
|    |               |             | Methods —<br>non-constructo<br>r method count                                  | Structural             |
| 8  | nof           | int         | Number of<br>Fields — all<br>fields<br>regardless of<br>visibility             | Extended<br>Structural |
| 9  | cida          | float [0,1] | Class Interface<br>Design<br>Assessment —<br>public methods<br>/ total methods | Extended<br>Structural |
| 10 | coa           | int         | Class<br>Ownership<br>Authority —<br>distinct git<br>commit authors            | Extended<br>Structural |
| 11 | jDeodorantTop | float [0,1] | Peak smell<br>signal from<br>jDeodorant<br>across all smell<br>types           | Expert Signal          |
| 12 | iPlasmaTop    | float [0,1] | Peak smell<br>signal from<br>iPlasma                                           | Expert Signal          |
| 13 | designniteTop | float [0,1] | Peak smell<br>signal from<br>Designite                                         | Expert Signal          |
| 14 | jspIritTop    | float [0,1] | Peak smell<br>signal from<br>JSpIRIT                                           | Expert Signal          |
| 15 | codeChum      | float [0,1] | Normalised<br>lines<br>added+deleted<br>over the<br>commit<br>window           | Evolutionary           |

|    |              |             |                                                                    |                   |
|----|--------------|-------------|--------------------------------------------------------------------|-------------------|
| 16 | volatility   | float [0,1] | Coefficient of variation of commit frequency                       | Evolutionary      |
| 17 | bugFixRatio  | float [0,1] | bug-fix commits / max bug-fix commits in project                   | Evolutionary      |
| 18 | authorCount  | int         | Distinct committers to this file                                   | Evolutionary      |
| 19 | expertScore  | float [0,1] | $E_{\square}$ — accuracy-weighted mean of the four expert signals  | Derived Composite |
| 20 | hotspotScore | float [0,1] | $H_{\square}$ — churn $\times$ bug-fix density (git hotspot proxy) | Derived Composite |
| 21 | cmScore      | float [0,1] | $CM_{\square}$ — normalised change magnitude from git history      | Derived Composite |

**Table #. Feature Vector**

### 3.4.2. Ensemble Detection through Multi-Detector Signal Fusion

The proposed system employs an ensemble-style detection aggregation framework that combines rule-inspired detection signals derived from multiple established code smell detection strategies. Unlike a conventional Mixture-of-Experts architecture in which multiple independently trained predictive models are dynamically routed through a gating mechanism,

the proposed framework uses detector-derived signals and structural software metrics as unified input features for a Random Forest classifier. The Random Forest model estimates the confidence and prioritization likelihood of detected smells, allowing the system to support explainable and context-aware refactoring recommendations.

## 1. Gating Network Mechanism

### **x (Input Vector)**

The feature vector  $v_n$  derived from the code element.

### **H(x) (Trainable Action Vector)**

An intermediate activation vector computed to introduce robustness through controlled noise:

$$H(x)_i = (x \cdot Wg)_i + \text{StandardNormal}() \cdot \text{Softplus}((x \cdot W\text{noise})_i)$$

## Equation 2. Gating Network Mechanism

### **TopK (Filtering Function)**

A function that retains only the top  $k$  highest activation values in the vector while setting all remaining values to negative infinity ( $-\infty$ ). This ensures that only the most confident expert toolsets contribute to the final decision, preventing low-confidence tools from diluting detection accuracy.

### **k (Activation Threshold)**

A hyperparameter specifying the number of expert toolsets considered per decision.

Limiting  $k$  enforces sparse activation, resulting in improved efficiency and more precise expert selection compared to dense aggregation.

### **Softmax (Normalization Function)**

A normalization function that transforms the filtered activation values into a probability

distribution whose elements sum to 1. This produces a clear, weighted ranking of expert tool suitability for each detected code smell.

## 2. Accuracy Matrix

The final stage of the ensemble detection pipeline utilizes an Accuracy Matrix to resolve tool conflicts and provide high-confidence detection results. This step implements the decision logic established in iSMELL (Wu et al., 2024), which bridges the "Confidence Gap" by ensuring detection is dominated by the most statistically relevant expert for each specific smell type.

**Classification Mechanism:** The system performs a dot product operation between the gating weight vector ( $G\sigma(x)$ ) and an expert accuracy matrix ( $M \in R^{4 \times 3}$ ) to produce a weighted accuracy matrix ( $M_{wei}$ ).

$$M_{wei} = G\sigma(x) \cdot M$$

### Equation 3. Accuracy Matrix

(Source: Wu et al., 2024)

#### Variables and Rationale:

- **Expert Accuracy Matrix ( $M \in R^{4 \times 3}$ ):** A matrix where each of the 4 rows represents a specialized expert toolset (iPlasma, DesigniteJava, JDeodorant, and JSpIRIT) and each of the 3 columns represents a core code smell category (God Class, Feature Envy, and Long method).
- **Weighted Confidence ( $M_{wei}$ ):** This matrix contains column vectors that compare the weighted confidence of all expert toolsets for each specific smell type. By weighting the accuracy of expert outputs, the system ensures that those toolsets with higher recognition ability for specific structural patterns have a greater impact on the final decision.

- **Final Detection Outcome:** In the final decision stage, the system identifies the expert tool with the highest weighted accuracy from the corresponding column vectors . This selected expert's finding is then used as the definitive result to confirm the presence of the corresponding code smell .

### 3.4.3. Refactoring Impact Simulation

Before a recommendation is finalized, the system performs a Refactoring Impact Simulation to estimate the Return on Investment (ROI) of the proposed change. This stage addresses the "Actionability Gap" by ensuring that only those refactorings that yield a measurable structural gain are prioritized.

The simulation performs a "virtual" application of a refactoring (e.g., Extract Method or Move Method) on the Abstract Syntax Tree (AST). This process follows a pre/post structural metric estimation approach:

#### 1. Baseline Extraction

The system records the current CK metrics ( $M_{current}$ ) of the targeted code element.

#### 2. AST Transformation

The system simulates the code modification by creating a temporary, in-memory version of the AST.

#### 3. Delta Calculation ( $\Delta$ )

The system re-calculates the metrics for the simulated structure ( $M_{simulated}$ ) and determines the net improvement:

$$\Delta_n = \sum_{i=1}^m (M_{current,i} - M_{simulated,i})$$

#### Equation 4. Delta Calculation ( $\Delta$ )

**Estimated Structural Improvement ( $\Delta_n$ ):** The final  $\Delta_n$  value represents the aggregate

improvement in structural indicators

#### 3.4.4. Weighted Multi-Criteria Analysis (WMCA)

Following detection, the system addresses the Prioritization Problem by calculating a Prioritization Factor (Pn) for each identified smell. This module utilizes the weighted multi-criteria framework proposed by Islam, Maruf, and Černý (2022) to rank refactoring candidates by synthesizing their strategic structural importance, dynamic runtime frequency, and historical evolutionary risk

$$P_{total} = \left[ (\beta(\alpha \frac{E_n}{E_{max}} + (1 - \alpha) \frac{H_n}{H_{max}}) + (1 - \beta) \frac{CM_n}{CM_{max}}) \times \frac{CL_n}{CL_{max}} \times CS \right] + w_1 \frac{SR_n}{SR_{max}} + w_2 \frac{\Delta_n}{\Delta_{max}}$$

#### Equation 5. Weighted Multi-Criteria Analysis (WMCA)

Variable Definitions and Rationale

##### **E□ (Strategic Importance)**

Measures the number of incoming and outgoing edges in the business process graph. High centrality identifies load-bearing components, where refactoring produces broader architectural impact across the system.

##### **H□ (Runtime Impact)**

Represents execution frequency derived from application log analysis. This metric identifies “hot paths” where structural code smells impose the greatest runtime performance cost.

E□ represents the level of agreement among expert-derived detection signals. It measures how consistently the implemented rule-based detectors or tool-inspired strategies identify the

same code smell. A higher  $E_{ij}$  value indicates stronger agreement among detection signals and increases the prioritization score.

$H_{ij}$  represents the historical modification activity of the affected code element based on available version-control data. Code elements with higher modification frequency are treated as hotspots and are given higher priority because they are more likely to be revisited or changed. If version history is unavailable,  $H_{ij}$  may be assigned a neutral default value or omitted from the scoring function.

#### **CM<sub>ij</sub> (Code Evolution)**

Represents the number of historical modifications extracted through repository mining using JGit. High modification frequency indicates high volatility, signaling regions more likely to accumulate technical debt.

**CL<sub>n</sub> (Criticality Level)** The aggregated severity of the detected code smell as reported by the detection experts. This ensures that critical structural issues are prioritized over minor stylistic concerns.

#### **CS (Confidence Score)**

A probabilistic output generated by the Mixture-of-Experts (MoE) gating network. This score ensures that only high-confidence detections receive high prioritization rankings.

#### **SR<sub>n</sub> (Security Risk)**

Quantifies violations of information hiding principles using the Class Object Access (COA) and Class Internal Data Access (CIDA) metrics. This component addresses potential safety gaps in system design.

#### **$\Delta n$ (Structural Improvement)**

Represents the projected refactoring return on investment (ROI) calculated during the simulation phase. The value aggregates predicted improvements across structural metrics including  $\Delta LOC$ ,  $\Delta CC$ ,  $\Delta CBO$ , and  $\Delta LCOM$ .

#### **$\alpha, \beta, w_1, w_2$ (Tunable Parameters)**

Weighting parameters that balance runtime relevance, long-term evolutionary risk, structural severity, and security considerations. These parameters are typically configured within the range 0.4–0.6, depending on project priorities and system context.

### **3.4.5. Deterministic Context-Aware Sequencing (DCAS)**

To address the refactoring ordering concern discussed by Maini, Kaur, and Kaur (2024), the system applies a deterministic sequencing heuristic called Deterministic Context-Aware Sequencing (DCAS). DCAS is designed to organize multiple refactoring recommendations into a more coherent order by considering both priority scores and structural dependency relationships. It does not claim to solve the general refactoring ordering problem or guarantee build safety.

#### **Tier 1: Priority-Based Sorting**



The first tier performs an initial prioritization of refactoring candidates using the total WMCA prioritization score  $P_{total}$ . Candidates with higher prioritization values are treated as structurally significant and are considered earlier during graph construction. Formally,

$$P_{total} = \sum_{i=1}^n w_i x_i$$

**Equation 6. name**

where  $X_i$  represents the normalized prioritization criteria and  $W_i$  represents the corresponding criterion weight. The resulting ordered candidate set serves as the baseline sequence before dependency constraints are applied.

## **Tier 2: Smell-Semantic Edge Injection**

The second tier augments the dependency graph using a static smell-type interaction rule table. Unlike ordinary dependency edges derived from source code structure, these edges encode semantic ordering constraints derived from refactoring theory and code smell interaction behavior. The algorithm inserts forced predecessor edges into the directed graph before topological sorting is executed.

For example, if a God Class is the target of a Feature Envy relationship, the structural decomposition of the God Class must occur before relocating methods associated with Feature Envy. Similarly, Long Method decomposition must precede Feature Envy correction when the envious behavior originates from the complex method itself.

The resulting directed graph is therefore composed of both structural dependency edges and smell-semantic precedence edges.

| Condition                                                                           | Action / Forced Edge                                           | Rationale                                                                                  |
|-------------------------------------------------------------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Class A is detected as God Class and Class B exhibits Feature Envy toward A         | $A \rightarrow B$                                              | Structural decomposition of the God Class must occur before relocating dependent methods.  |
| Class A contains a Long Method and Class B exhibits Feature Envy toward that method | $A \rightarrow B$                                              | Method extraction reduces complexity before cross-class relocation is attempted.           |
| Two God Class candidates belong to the same inheritance hierarchy                   | Shallower DIT class precedes deeper DIT class                  | Refactoring higher-level abstractions first minimizes inheritance ripple effects.          |
| Multiple smells occur within the same class                                         | God Class $\rightarrow$ Feature Envy $\rightarrow$ Long Method | Structural extraction modifies the class architecture that later refactorings depend upon. |

**Table #. Sequence Diagram of the System**

### **Tier 3: Intra-Class Conflict Resolution**

The third tier resolves conflicts involving multiple refactoring candidates targeting the same class. When two or more smells share the same qualified class name, DCAS applies a fixed precedence hierarchy to avoid contradictory structural transformations. The precedence hierarchy is defined as:

*God Class  $\rightarrow$  Feature Envy  $\rightarrow$  Long Method*

This ordering is based on structural dependency relationships among refactoring operations. Extract Class operations associated with God Class refactoring modify the overall architecture of the class and therefore must precede Move Method operations associated with Feature Envy.

Likewise, Long Method refactoring is deferred until broader structural decomposition has stabilized the class organization.

To operationalize this mechanism, DCAS inserts internal precedence edges among candidates sharing the same qualified class identifier. These edges are treated as mandatory ordering constraints during subsequent graph traversal.

#### **Tier 4: Topological Sorting with Deterministic Cycle-Breaking**

The final tier performs topological sorting over the augmented dependency graph using a modified Kahn-based traversal algorithm. The graph is modeled as a directed acyclic graph (DAG), where vertices represent refactoring candidates and edges represent structural or semantic precedence constraints.

Candidates are grouped according to DAG depth level during traversal. Let  $L(v)$  denote the topological level assigned to node  $v$ . Candidates with equal depth values satisfy:

$$L(v_i) = L(v_j)$$

**Equation 7. name**

and are considered parallelizable because no direct dependency edge exists between them. This enables coordinated refactoring execution among multiple developers while preserving deterministic ordering semantics.

In cases where cyclic dependencies are detected, DCAS applies a deterministic cycle-breaking heuristic rather than rejecting the recommendation set. When Kahn's algorithm stalls due to

residual non-zero in-degree nodes, the algorithm evaluates all conflicting edges and removes the edge whose connected candidates exhibit the smallest absolute prioritization difference:

$$\Delta P_{total} = |P_{total}(v_i) - P_{total}(v_j)|$$

**Equation 8. name**

The edge associated with the minimum  $\Delta P_{total}$  value is removed because the relative ordering of these candidates has the least prioritization significance. After edge removal, topological sorting resumes until all remaining nodes are sequenced.

This deterministic resolution strategy allows DCAS to preserve sequencing continuity while minimizing disruption to the original prioritization ordering. Removed edges are recorded as soft sequencing conflicts rather than hard execution failures, thereby maintaining developer visibility into the heuristic adjustment.

Overall, the updated DCAS framework extends beyond conventional dependency sorting by integrating smell semantics, precedence-aware conflict management, DAG-level parallelism, and deterministic cycle-resolution heuristics into a unified sequencing model for refactoring recommendation planning.

**Tier 1: Greedy Value Sorting**

All detected smells are initially sorted in descending order of their  $P_{\square}$  scores using an  $O(n \log n)$  greedy sort. This phase ensures that refactorings with the highest estimated ROI are addressed first.

## **Tier 2: Structural Heuristic Filtering**

The sorted list is then refined using rule-based constraints derived from the system's dependency graph.

- **Topological Reordering:**

Parent-child dependencies are identified (e.g., *Extract Method* must precede *Move Class*). When conflicts arise, the sequence is automatically adjusted so that foundational refactorings precede dependent ones.

- **Behavioral Safety:**

This deterministic reordering is intended to reduce the likelihood of ordering-related conflicts by organizing refactoring recommendations according to structural dependencies. By prioritizing dependency-aware adjustments, the approach helps minimize situations where earlier refactoring actions may negatively affect subsequent recommendations. However, the method does not guarantee build safety or behavioral preservation, as final validation depends on IntelliJ's refactoring precondition checks, successful compilation, and developer judgment.

- **Efficiency Consideration**

By avoiding quadratic-time search-based metaheuristics (e.g., HSHEP), the DCAS approach remains lightweight and time-efficient, making it suitable for real-time execution within an IDE.

## **Deterministic Context-Aware Sequencing (DCAS)**

To address the refactoring ordering concern discussed by Maini, Kaur, and Kaur (2024), the system applies a deterministic sequencing heuristic called **Deterministic Context-Aware Sequencing (DCAS)**. DCAS is designed to organize multiple refactoring recommendations into a more coherent order by considering both priority scores and structural dependency relationships. It does not claim to solve the general refactoring ordering problem or guarantee build safety.

### **Tier 1: Priority-Based Sorting**

All detected refactoring candidates are initially sorted in descending order according to their computed prioritization scores. This step increases the likelihood of candidates with higher estimated relevance, severity, or expected structural benefit are considered earlier in the recommendation list. The sorting process follows an  $O(n \log n)$  ordering procedure.

### **Tier 2: Structural Dependency Adjustment**

After the initial priority-based sorting, the system refines the recommendation order using rule-based constraints derived from the dependency graph of the analyzed Java code. The dependency graph identifies relationships among affected classes, methods, fields, and call references. When the system detects that one recommendation may affect the validity or context of another recommendation, the order is adjusted so that foundational or dependency-sensitive changes are presented first.

For example, if a method-level refactoring affects a method that is also involved in a class-level refactoring, the system may prioritize the lower-level refactoring first or flag the relationship for manual review. Topological ordering is used only as one component of this adjustment process and is applied when clear dependency relationships can be represented as directed edges.

### **Conflict Handling and Safety Limitation**

DCAS is intended to reduce obvious ordering conflicts, not to guarantee behavioral preservation. If the system identifies dependency cycles, unresolved references, or conflicting recommendations, the affected recommendations are flagged for manual review rather than being presented as automatically safe. Therefore, DCAS provides a deterministic and explainable ordering strategy, but final validation still depends on IntelliJ's refactoring precondition checks, successful compilation, and developer judgment.

### **Efficiency Consideration**

Because DCAS uses deterministic sorting and dependency-based adjustment rather than computationally expensive search-based metaheuristics, it remains lightweight enough for use in an IDE-based recommendation workflow. However, its purpose is recommendation ordering, not exhaustive optimization of all possible refactoring sequences.

## **3.5. Hardware and Software Specifications**

The hardware and software requirements needed to support the development and deployment phases of the proposed system are as follows:

The specifications below outline the requirements needed to support the Development and Training Environment of the proposed system, which entails the development, implementation, compilation, and training of the system using the Random Forest Mixture of Experts approach.

The specifications below outline the requirements needed to support the Target Deployment Environment of the proposed system, which entails the development, implementation, compilation, and training of the system using the Random Forest Mixture of Experts approach.

Providing the specifications of the system enables the development of the system, the training of the model, as well as the deployment of the system in real-world usage scenarios.

**3.5.1. Hardware Specifications**

**3.5.1.1. Development Environment (Experimental Setup)**

The development and test phases required a high-performance workstation. This was a specialized environment that was developed to handle computationally intensive operations, which included compiling the IntelliJ SDK, mining version control data using JGit, and training the machine learning classifier offline.

| Component | Description                   | Minimum Requirement            | Recommended Specification      |
|-----------|-------------------------------|--------------------------------|--------------------------------|
| Processor | Central Processing Unit (CPU) | Quad-core (Intel i5 / Ryzen 5) | Octa-core (Intel i7 / Ryzen 7) |
| RAM       | System Memory                 | 2 GB                           | 8 GB                           |
| Storage   | Internal Drive                | 256 GB SSD                     | 512 GB NVMe SSD                |



|                |                    |                                            |                                  |
|----------------|--------------------|--------------------------------------------|----------------------------------|
| <b>Display</b> | Monitor Resolution | 1024×768                                   | 1920×1080                        |
| <b>OS</b>      | Operating System   | Windows 10 (64-bit)<br>macOS 12.0 or later | Latest Version of Windows, macOS |

**Table 1. Hardware Specifications for Researchers (Development & Training)**

### 3.5.1.2. Deployment Environment ( End-User Requirements)

Considering the hardware requirements from the point of view of the end-users, who are Java programmers, it is found that the requirements match the standard recommendations to run the IntelliJ IDEA platform. Since the Machine Learning model is pre-trained and packaged locally within the plugin, the model is very light; hence, the end-user does not need a heavy computing machine to work on. Nevertheless, the machine must be sufficiently equipped to allow the plugin's real-time analysis to not interfere with the performance of the IDE.

| <b>Component</b> | <b>Description</b>            | <b>Minimum Requirement</b> | <b>Recommended</b>               |
|------------------|-------------------------------|----------------------------|----------------------------------|
| Processor        | Central Processing Unit (CPU) | Quad-core CPU              | Intel Core i5 / Ryzen 5          |
| RAM              | System Memory                 | 8 GB                       | 16 GB                            |
| Storage          | Internal Drive                | 3.5 GB HDD                 | SSD (SATA/NVMe)                  |
| Display          | Monitor Resolution            | 1024×768                   | 1920 × 1080                      |
| OS               | Operating System              | Windows 10                 | Latest Version of Windows, macOS |

**Table 2. Recommended Hardware for End-Users (Deployment)**

### **3.5.2. Software Requirements**

The software tools, frameworks, and technologies used in the development, analysis, and execution of the proposed system are presented in this part of the study. The software architecture of the proposed system has two major components, which are the development environment used by the researchers to implement the system, and the deployment environment used by the end-users to execute the system.

#### **3.5.2.1. Development and IDE Environment**

The following technologies were utilized to engineer the IntelliJ plugin, extract structural and evolutionary metrics, and train the Machine Learning models offline:

- **Java Development Kit (JDK 25)**

Served as the primary programming language, development, and runtime environment. It is strictly required for core plugin development and ensures compatibility with IntelliJ's underlying mechanisms.

- **IntelliJ IDEA SDK**

A specialized software development kit provided by JetBrains. It supplied the necessary Application Programming Interfaces (APIs) to construct the plugin's graphical User Interface (UI), including the custom tool windows and the Recommendations Pane

- **IntelliJ Program Structure Interface (PSI)**

An internal API that generates a structural representation of the source code. It was programmatically utilized to execute recursive traversals of the Abstract

Syntax Tree (AST), facilitating the extraction of structural CK metrics such as WMC, CBO, and LCOM.

- **JGit**

A lightweight, pure Java library implementing the Git version control system was used, which was incorporated into the automated data extraction pipeline to programmatically mine the repository history of the analyzed Java projects. This was used to extract critical evolutionary metrics, such as code churn, which are heavily utilized by the WMCA engine to determine historical risk.

- **Expert Tools**

A specialized suite of established static analysis and refactoring detection utilities used to generate ground-truth labels and baseline detection signals. These tools provide the initial structural evaluations that are fed into the Random Forest Gating Network, allowing the MoE architecture to dynamically weigh their individual accuracies and resolve conflicting code smell detections.

### **3.5.2.2. Deployment Software (End-User Environment)**

- **IntelliJ IDEA (Host IDE)**

The integrated development environment functioning as the primary host platform. This is where the plugin is installed, executed, and where developers interact with the prioritized refactoring recommendations.

- **Maven and Gradle**

Industry-standard build automation and dependency management tools. These must be present in the target projects analyzed by the plugin to guarantee

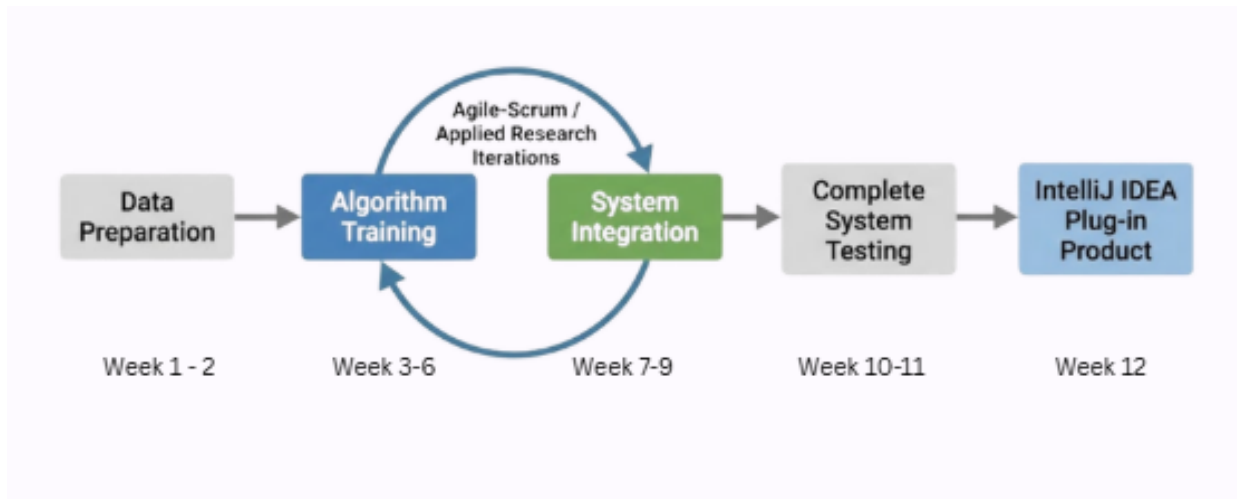
successful code compilation and accurate symbol resolution during live metric extraction.

| Software / Technology                                | Description                        | Primary Purpose                              | Deployment Phase         |
|------------------------------------------------------|------------------------------------|----------------------------------------------|--------------------------|
| <b>JDK 25</b>                                        | Java runtime environment           | Core plugin development and execution        | Development & Deployment |
| <b>IntelliJ IDEA SDK</b>                             | Plugin development kit             | Access to IDE APIs and UI integration        | Development              |
| <b>IntelliJ PSI</b>                                  | Source code API                    | AST traversal and CK metric extraction       | Development & Deployment |
| <b>JGit</b>                                          | Java-based Git library             | Extraction of evolutionary metrics (churn)   | Development              |
| <b>Python (v3.14)</b>                                | Programming language               | Offline model training environment           | Development              |
| <b>scikit-learn</b>                                  | Machine learning library           | Training the Random Forest classifier        | Development              |
| <b>Baseline Tools</b><br>( <i>JDeodorant, etc.</i> ) | Static Analysis Tools              | Expert logic reference for validating smells | Development              |
| <b>IntelliJ IDEA</b>                                 | Integrated Development Environment | Host platform for plugin execution and UI    | Deployment (End-User)    |
| <b>Maven / Gradle</b>                                | Build automation tools             | Dependency resolution for analyzed projects  | Deployment (End-User)    |

**Table 3. Summary of Development and Deployment Software Specifications**

### 3.6. Method in Developing Software

The system development process used an Agile Scrum framework together with an organized applied research approach to enable continuous design evolution and system improvements. The workflow began with data preparation and then advanced through three main stages: algorithm training and system integration and complete system testing. The process focused on developing a working software product that functioned as an IntelliJ IDEA plug-in.



**Figure 6. Agile-Scrum Development Process**

### 3.7. Method in Evaluation of the Study

To evaluate the proposed IntelliJ-based refactoring decision-support system, the study adopts a multi-layered evaluation framework that examines both system-level quality and algorithmic performance. At the system level, evaluation is guided by the ISO/IEC 25010 Product Quality Model (PQM) to assess selected software quality characteristics relevant to usability and functional suitability. At the algorithmic level, the effectiveness of the detection and prioritization

mechanisms is measured using classification metrics derived from confusion matrix analysis, as well as ranking-based information retrieval metrics.

This combined approach allows the study to assess both the usability of the developed tool and the performance of its underlying decision-support mechanisms. However, the evaluation is conducted within a limited scope, focusing on selected Java code smells and primarily involving student and novice users. Therefore, the results should be interpreted as indicative of system effectiveness within the defined context rather than as a comprehensive validation across all software development scenarios.

#### **3.7.1. Before-and-After Structural Evaluation**

In addition to product-level and algorithmic evaluation, the study includes a before-and-after structural evaluation to determine whether the recommended refactoring alternatives produce measurable changes in source code structure. This evaluation is necessary because classification and ranking metrics can determine whether the system detects and ranks recommendations effectively, but they do not directly show whether applying the recommendations improves the affected code elements.

For each selected Java project, the system first records baseline structural metrics before any refactoring recommendation is applied. These baseline metrics include Lines of Code (LOC), Cyclomatic Complexity (CC), Coupling Between Objects (CBO), Lack of Cohesion in Methods (LCOM), and Weighted Methods per Class (WMC). After the system generates prioritized refactoring recommendations, the selected top-ranked recommendation is applied using IntelliJ's built-in refactoring support or manually implemented according to the

recommended refactoring alternative. The same structural metrics are then recalculated after refactoring.

**The before-and-after change is computed as:**

$$\Delta \text{Metric} = \text{Metric\_before} - \text{Metric\_after}$$

A positive  $\Delta$  value indicates a reduction in the measured structural issue, while a zero or negative value indicates no improvement or possible structural regression. The results are interpreted as observed structural changes within the selected Java projects, not as proof of universal behavioral correctness or long-term maintainability improvement.

To ensure that the proposed prioritization approach is not evaluated in isolation, the before-and-after results will also be compared with baseline ranking approaches, such as severity-only ranking, confidence-only ranking, rule-only ranking, and random ranking. For each method, the same number of top-ranked recommendations will be applied or evaluated, and the resulting metric changes will be compared.

| <b>Metric</b> | <b>Before<br/>Refactoring</b> | <b>After<br/>Refactoring</b> | <b><math>\Delta</math> Change</b> | <b>Interpretation</b>                                             |
|---------------|-------------------------------|------------------------------|-----------------------------------|-------------------------------------------------------------------|
| LOC           | value                         | value                        | before –<br>after                 | Lower value may indicate reduced size or extracted responsibility |

|      |       |       |                   |                                                          |
|------|-------|-------|-------------------|----------------------------------------------------------|
| CC   | value | value | before –<br>after | Lower value may indicate reduced control-flow complexity |
| CBO  | value | value | before –<br>after | Lower value may indicate reduced coupling                |
| LCOM | value | value | before –<br>after | Lower value may indicate improved cohesion               |
| WMC  | value | value | before –<br>after | Lower value may indicate reduced class responsibility    |

### 3.7.2. System Validation and Evaluation Metrics

To evaluate the effectiveness of the proposed system, this study validates both the Random Forest model used in the Mixture-of-Experts framework and the refactoring prioritization mechanism. Random Forest is an ensemble learning algorithm that constructs multiple decision trees using bootstrap sampling and aggregates their predictions through majority voting, which improves predictive accuracy and reduces overfitting (Breiman, 2001). In this study, the model is trained using labeled datasets containing known code smells and detection outputs from multiple code smell detection tools. This allows the model to learn patterns that indicate when a tool is likely to produce a correct detection.



To ensure that the model generalizes to unseen software systems, the study applies cross-project validation, where the Random Forest model is trained on several software projects and tested on projects that were not included during training. This approach helps verify that the model can reliably predict tool correctness across different software systems.

The performance of the model in detecting code smells is evaluated using confusion matrix-based classification metrics, which compare the predicted results against the ground truth dataset. The following metrics are used:

Precision measures the proportion of correctly predicted positive instances among all predicted positives and is calculated as:

$$Precision = \frac{TP}{TP + FP}$$

where TP (True Positives) represents correctly predicted smell detections and FP (False Positives) represents incorrect smell detections.

#### **Equation 9. Precision**

Recall measures the proportion of correctly predicted positive instances among all actual positives and is defined as:

$$Recall = \frac{TP}{TP + FN}$$

### Equation 10. Recall

where FN (False Negatives) represents cases where the model fails to detect an existing code smell.

The F1-score balances precision and recall and is calculated as:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

(Where TP = True Positive, FP = False Positive, FN = False Negative)

### Equation 11. F1-Score

These metrics collectively measure how accurately the system detects code smells while balancing detection accuracy and completeness.

In addition to classification performance, the study evaluates the quality of probability predictions generated by the Random Forest model using the Brier Score, which measures the difference between predicted probabilities and actual outcomes:

$$Brier\ Score = \left(\frac{1}{N}\right) \cdot \sum (\pi - yi)^2$$

### Equation 12. Brier Score

where  $\pi$  represents the predicted probability and  $y_i$  represents the actual outcome (0 or 1). A lower Brier Score indicates better calibrated probability predictions, meaning the reliability estimates of the detection tools are more accurate.

To evaluate the effectiveness of the refactoring prioritization process, the study uses Information Retrieval ranking metrics to measure how well the system ranks recommended refactoring actions. The following ranking metrics are used:

Precision at K ( $P@K$ ) measures the proportion of relevant refactoring recommendations within the top K results returned by the system. This metric evaluates whether the most important refactoring suggestions appear in the highest ranked positions.

Normalized Discounted Cumulative Gain at K ( $NDCG@K$ ) evaluates the overall quality of the ranking produced by the system. This metric penalizes relevant recommendations that appear lower in the ranking and rewards those placed at higher positions.

$$NDCG@K = \frac{DCG@K}{IDCG@K}$$

(Where DCG represents the Discounted Cumulative Gain and IDCG represents the Ideal ranking)

**Equation 13.  $NDCG@K$**

The implementation and evaluation of the Random Forest model are conducted using the scikit-learn machine learning library, which provides tools for training Random Forest classifiers and computing evaluation metrics such as precision, recall, F1-score, and Brier score.

### **ISO 25010:2023 Product Quality Model (PQM)**

The primary basis for evaluating the completed system is the ISO/IEC 25010:2023 Product Quality Model. ISO 25010 provides a structured and internationally recognized framework for assessing software systems across diverse development environments. Its Product Quality Model defines eight essential characteristics: Functional Suitability, Reliability, Performance Efficiency, Interaction Capability, Security, Compatibility, Maintainability, and Flexibility.

The evaluation of Functional Suitability assesses whether the IntelliJ plugin can detect Java code smells while providing suitable refactoring options that match the system requirements. The evaluation of reliability determines whether the plugin maintains its operational status throughout continuous testing while it operates within the IntelliJ IDEA environment. Performance Efficiency measures the responsiveness and resource usage of the plugin while analyzing code to ensure it does not negatively affect IDE performance. The evaluation of Interaction Capability assesses how users can comprehend and navigate the plugin interface to use it for refactoring purposes. Security ensures that the plugin safely handles source code and project files without exposing, altering, or compromising user data. The assessment of Compatibility examines whether the plugin can work together with IntelliJ IDEA and different Java projects and all other installed plugins. The evaluation of Maintainability assesses how simple it is to change or enhance the plugin codebase for upcoming updates and new features. Flexibility measures the capability of the system to support future enhancements, such as additional code smell detections or adaptations to newer Java versions

Using ISO 25010 ensures that the system is evaluated not only in terms of technical correctness but also in terms of product-level quality and usability in practical software development contexts.

The primary basis for evaluating the completed system is the ISO/IEC 25010:2023 Product Quality Model. ISO 25010 provides a structured and internationally recognized framework for assessing software systems across diverse development environments. Its Product Quality Model defines eight characteristics: Functional Suitability, Reliability, Performance Efficiency, Interaction Capability, Security, Compatibility, Maintainability, and Flexibility.

In this study, ISO 25010 is used as a guiding framework to evaluate selected quality characteristics that are most relevant to the developed IntelliJ-based refactoring decision-support system. The evaluation primarily focuses on Functional Suitability, Interaction Capability, and Performance Efficiency, as these characteristics directly relate to the system's ability to provide usable and effective refactoring recommendations within an IDE environment.

Functional Suitability assesses whether the plugin can detect selected Java code smells and provide appropriate refactoring alternatives that align with the defined system objectives. Interaction Capability evaluates how users understand, navigate, and utilize the plugin interface when reviewing and selecting refactoring recommendations. Performance Efficiency examines the responsiveness of the plugin and its impact on IDE performance during code analysis.

Other characteristics such as Reliability, Security, Compatibility, Maintainability, and Flexibility are considered at a conceptual or design level. These aspects are discussed in terms of system design decisions, development constraints, and intended future improvements, rather than being exhaustively measured through empirical testing within the scope of this study.

Using ISO 25010 as a reference framework allows the study to structure the evaluation of system quality and usability in a consistent and recognized manner. However, the assessment is limited to selected characteristics and should be interpreted within the defined scope of the system and its intended use for student and novice developers.

### 3.7.3. Classification Metrics (Detection Performance)

In order to assess the performance of the Random Forest Mixture-of-Experts model in detecting code smells with respect to the Ground Truth, standard Confusion Matrix metrics are used in the following:

- **Precision:** Measures the accuracy of positive predictions

$$Precision = \frac{TP}{TP + FP}$$

#### Equation 14. Precision

- **Recall (Sensitivity):** Measures the system's ability to find all existing smells.

$$Recall = \frac{TP}{TP + FN}$$

#### Equation 15. Recall

- **F1-Score:** The harmonic mean of Precision and Recall, providing a single metric to evaluate the balance between detection completeness and accuracy.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

(Where TP = True Positive, FP = False Positive, FN = False Negative)

#### Equation 16. F1-Score

#### 3.7.4. Ranking and Prioritization Metrics

For measuring the effectiveness of the application of Weighted Multi-Criteria Analysis (WMCA) on the ordering of recommendations, Information Retrieval (IR) metrics are employed:

- **Precision at K (P@K):** It refers to the proportion of relevant items among the top K recommended items. This metric checks if the top k refactorings are relevant.
- **Normalized Discounted Cumulative Gain at K Positions (NDCG@K):** This metric is generally used to evaluate the quality of ranking. If some refactoring is placed at a lower position in the ranking, then the system gets penalized.

$$NDCG@K = \frac{DCG@K}{IDCG@K}$$

(Where DCG represents the Discounted Cumulative Gain and IDCG represents the Ideal ranking)

#### Equation 17. NDCG@K

#### 3.7.5. Before-and-After Refactoring Metrics

The before-and-after evaluation measures the structural effect of applying selected refactoring recommendations. Unlike the Refactoring Impact Simulation stage, which estimates possible metric improvements before a recommendation is finalized, this evaluation records the actual structural metrics before and after the selected refactoring is applied.

The evaluated metrics include LOC, Cyclomatic Complexity, CBO, LCOM, and WMC. These metrics were selected because they correspond to the types of structural issues targeted by the system, particularly Long Method, Feature Envy, and God Class. The comparison allows the study to determine whether the top-ranked recommendations produce measurable structural improvements in the tested Java projects.

The observed metric delta is calculated using:

$$\Delta\text{Metric} = \text{Metric\_before} - \text{Metric\_after}$$

**Equation 18.  $\Delta\text{Metric}$**

A positive delta suggests improvement for metrics where lower values are preferable. However, the interpretation of results depends on the smell type and the affected code element. For example, a reduction in method LOC and Cyclomatic Complexity is most relevant for Long Method, while changes in CBO and LCOM are more relevant for Feature Envy and God Class.

This evaluation does not claim to prove complete behavioral preservation. Instead, it measures structural improvement after applying selected recommendations, while final correctness remains dependent on successful compilation, IntelliJ refactoring checks, and developer review.

### **3.7.6. Evaluation Bias Mitigation**

To strengthen the technical validity of the study and reduce possible evaluation bias, the researchers incorporated an Expert Validation Procedure as part of the evaluation framework.



The study acknowledges that while both IT/CS students and professionals can provide valuable usability and user experience feedback, novice respondents may have limited experience in software maintenance and refactoring assessment. Because of this, additional expert-based validation is necessary to assess the technical appropriateness of the generated recommendations.

In this study, both IT/CS students and professionals will use the system and answer the evaluation survey. However, professionals with relevant software engineering experience will additionally participate in a technical validation process to assess the correctness and practical relevance of the generated recommendations.

## **Purpose of Expert Validation**

The Expert Validation Procedure aims to:

1. Evaluate the technical correctness of the generated refactoring recommendations;
2. Assess whether the prioritization aligns with practical software maintenance considerations;
3. Reduce dependence on purely perception-based evaluation;
4. Supplement usability feedback with professional software engineering judgment; and
5. Improve the reliability and credibility of the evaluation framework.

## **Expert Evaluators**

The expert evaluators will consist of IT and Computer Science professionals with experience in software development, software maintenance, code review, or software engineering instruction.

These professionals will not only use the system and answer the general evaluation survey but will also perform an additional technical assessment of the generated recommendations.

## Expert Evaluation Process

A selected subset of the generated refactoring recommendations will be reviewed by professional evaluators. The recommendations will include:

- detected code smells,
- prioritization rankings,
- contextual explanations, and
- recommended refactoring actions.

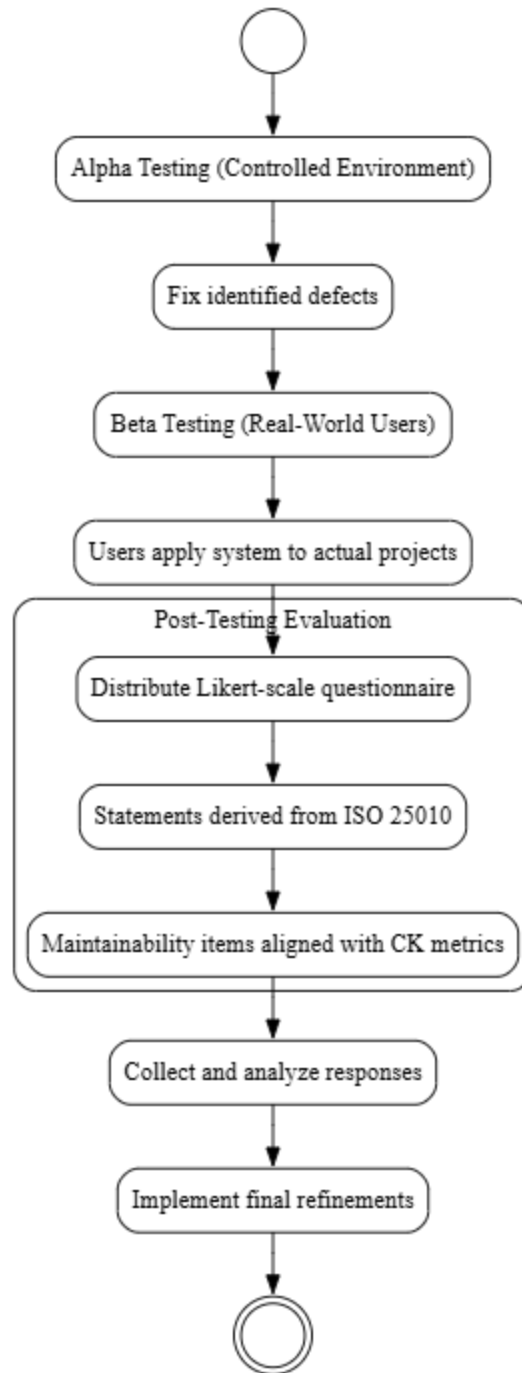
The expert evaluators will classify each recommendation according to the following categories:

| Classification        | Description                                                                                        |
|-----------------------|----------------------------------------------------------------------------------------------------|
| Appropriate           | The recommendation is technically valid and beneficial for maintainability improvement.            |
| Partially Appropriate | The recommendation contains valid aspects but may require adjustments or additional context.       |
| Inappropriate         | The recommendation is unnecessary, misleading, incorrectly prioritized, or technically unsuitable. |

### **3.8. Testing Method**

#### **3.8.1. Alpha and Beta Testing**

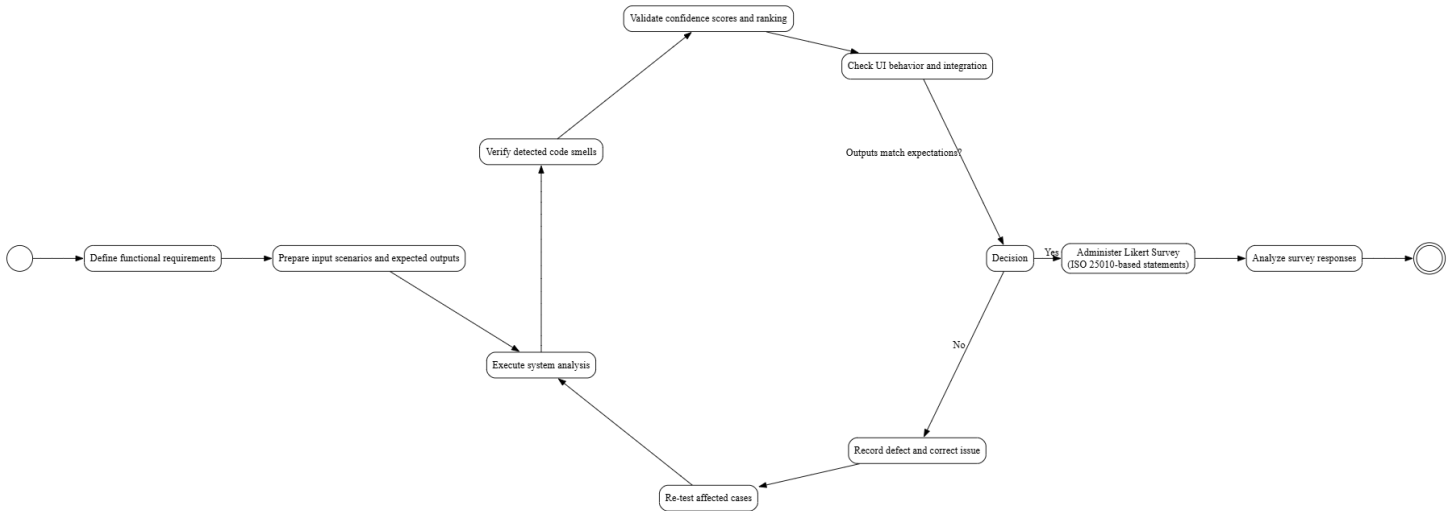
Alpha testing will be conducted in a controlled environment by the research team and selected technical evaluators to identify functional defects, integration issues, and computational inconsistencies prior to external release. Beta testing will follow, involving selected external users applying the system in real development environments to evaluate practical usability, performance under actual workloads, and recommendation usefulness. After completing system usage, both alpha and beta testers will respond to a structured Likert-scale questionnaire. Survey statements will be formulated based on ISO 25010 quality characteristics and CK-based maintainability indicators to measure perceived system quality, maintainability support, usability, and effectiveness. Collected responses will be statistically analyzed to determine overall system acceptability and areas for refinement.



**Figure 7. Alpha and Beta Testing Process**

### 3.8.2. Black Box Testing Procedure

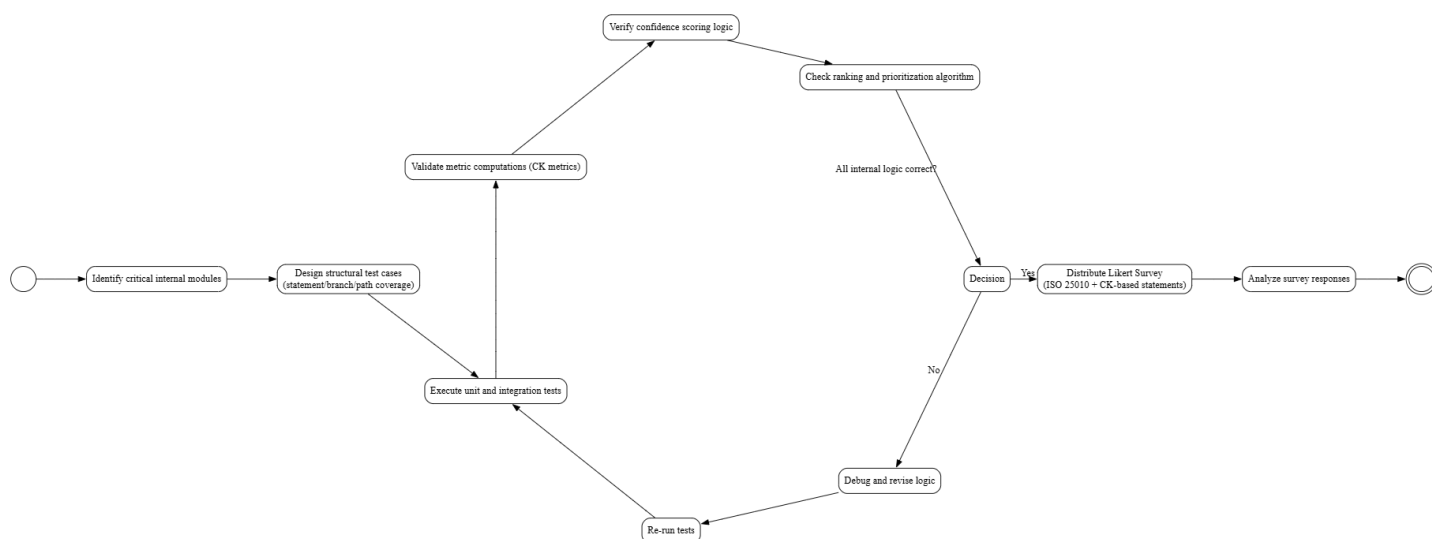
Black box testing will evaluate the system's functional behavior based on input-output validation without considering internal code structure. The system will be tested using controlled sample projects containing known code smells to verify detection accuracy, prioritization behavior, and interface responsiveness. Functional requirements such as smell



**Figure 8. Black Box Testing Flow**

### 3.8.3. White Box Testing Procedure

White box testing will focus on verifying internal processing logic, algorithm correctness, data flow integrity, and module interaction. Core components to be tested include the static analysis integration module, ensemble detection engine, Random Forest confidence estimator, and weighted multi-criteria prioritization mechanism. Control flow paths, decision branches, and data transformations will be examined to ensure accurate computation and proper sequencing of detection and prioritization processes.



### Figure 9. White Box Testing Flow

### 3.9. Data Collection Scheduling

[illegible]

**Table 5. Gantt Chart for Data Collection Scheduling**

Table 4 shows a Gantt chart that breaks down tasks into successive stages in order to decide how the data gathering will be managed. The system development and evaluation cycles establish the weekly time periods which define each development phase of the project. Data collection takes place during three core stages which include Baseline Acquisition, Algorithm Refinement and System Evaluation. Section 3.9 goes into additional detail about the methods used to collect the data for these activities.

### **3.10. Data Gathering**

The data gathering procedure for this study is divided into two distinct phases to satisfy the requirements of the machine learning architecture and the software evaluation phase. The first phase involves gathering static, pre-labeled datasets to train the Random Forest algorithm. The second phase involves sourcing live, real-world Java codebases to test the fully integrated IntelliJ IDEA plugin.

#### **3.10.1. Real-World Effectiveness Validation**

The system will undergo testing with existing open source Java projects to verify its operational performance in actual environmental conditions. The system will analyze actual codebases to produce refactoring suggestions which will be based on identified code smells and their respective priority assessment methods. The evaluation results will show whether the recommendations enhance code maintainability and help developers find essential refactoring opportunities.

### **3.10.2. Sources of Data**

#### **1. Training Datasets (Machine Learning Corpus)**

To train the algorithm to detect structural defects, the system utilizes the Machine Learning-based Code Quality (MLCQ) dataset as the primary training corpus, published by Madeyski and Lewowski (2020).

The system does not merely analyze the latest state of these repositories. Instead, it relies on a Python-based historical mining script utilizing the GitPython library to dynamically clone target repositories and traverse the project history back to the exact commit hash where the human expert identified the code smell. The raw extraction yields classes which the system then processes and cleans to extract a 23-feature Unified Feature Vector, mapping the dataset's legacy labels to three specific targets:

- God Class: Mapped from the dataset's legacy 'blob' label.
- Feature Envy: Extracted for method-level and class-level data access violations.
- Long Method: Extracted for cognitive and structural complexity thresholds.

#### **2. Testing Datasets (Live Java Codebases)**

In the absence of a pre-compiled test dataset to test the execution of real-time IDE plugins the evaluation phase will use live Java source codes as test data. These will be directly taken from open-source GitHub repositories. Repositories must be in Java and must use standard build automation tools (Maven or Gradle) to ensure that the plugin's Abstract Syntax Tree (AST) parser can execute correctly in the Black Box testing phase.



### 3.10.3. Dataset Preprocessing

Before feeding the raw mined data into the machine learning algorithm, strict preprocessing steps are mathematically applied via the offline Python pipeline to ensure data quality, prevent data leakage, and bridge the integration to the Java environment.

- **Data Reduction**

The raw datasets consist of some empty, artificial, or unanalyzable classes that cannot provide predictive value. The script programmatically drops any row from the matrix where both the Weighted Method Count (wmc) and Number of Methods (nom) are equal to zero. Instances which lack crucial features or characteristics that will help the prediction process are removed or filled, thus enhancing the raw data into an extremely robust training set.

- **Feature Scaling (Min-Max Normalization)**

To prevent structural metrics with massive numerical ranges from overshadowing smaller proportional metrics the 23 unified features undergo Min-Max Normalization. Furthermore, proportional metrics and expert confidence signals are mathematically clamped to enforce a strict [0.0, 1.0] boundary. The calculated scaling bounds are exported to a bounds.json dictionary. This bridging mechanism guarantees that live raw metrics scanned by the system are mathematically scaled identically before being passed to the ONNX inference engine.

- **Balancing the Dataset (SMOTE)**

In a real world software dataset an extreme class imbalance naturally exists where clean code vastly outnumbers smelly code. To prevent the “Accuracy Paradox” where the model biases toward the majority class, the Synthetic Minority Over-sampling Technique

(SMOTE) is applied. However, to prevent catastrophic data leakage, SMOTE is strictly isolated to the training fold only. The dataset is partitioned into 70% Training, 15% Validation, and 15% Testing segments. SMOTE utilizes the k-Nearest Neighbors (k-NN) algorithm to generate mathematically correct synthetic code smell instances strictly within the training fold's geometric bounds, ensuring the validation and test sets reflect true real-world distributions. This process is applied independently for each target label's training loop

#### 3.10.4. Respondent Survey Questionnaire

A structured survey questionnaire and interview guide were administered to selected respondents who evaluated the developed system. This will be distributed via google forms to the respondents after testing the system. The questionnaire utilized a five-point Likert scale to measure respondents' perceptions based on ISO 25010 quality characteristics.

| Statements                                                     | Responses       |
|----------------------------------------------------------------|-----------------|
| Data Privacy Policy Statement                                  | Multiple Choice |
| Personal Information                                           |                 |
| Full Name                                                      | Short Text      |
| I am a/an                                                      | Multiple Choice |
| Program                                                        | Short Text      |
| Professional Personal Information                              |                 |
| What is your current Job Title                                 | Short Text      |
| How many years of experience do you have in the tech industry? | Multiple Choice |
| Student Personal Information                                   |                 |
| Name of School                                                 | Short Text      |
| Program                                                        | Short Text      |

|                                                                                      |                 |
|--------------------------------------------------------------------------------------|-----------------|
| Year Level                                                                           | Multiple Choice |
| Functional Suitability                                                               |                 |
| The system accurately detects code smells in the source code                         | Likert Scale    |
| The refactoring recommendations are appropriate for the identified issues            | Likert Scale    |
| The prioritization of recommendations reflects their actual importance               | Likert Scale    |
| The system performs its intended functions completely and correctly                  | Likert Scale    |
| The system provides useful insights for improving code quality                       | Likert Scale    |
| Reliability                                                                          |                 |
| The system operates without unexpected crashes during background analysis            | Likert Scale    |
| The system produces consistent results across multiple analyses of the same codebase | Likert Scale    |
| The system maintains stable performance during extended use.                         | Likert Scale    |
| Performance Efficiency                                                               |                 |
| The system analyzes code within an acceptable time.                                  | Likert Scale    |
| The UI responsiveness of the system is satisfactory.                                 | Likert Scale    |
| The system does not excessively consume IDE memory or CPU resources during analysis  | Likert Scale    |
| Interaction Capability                                                               |                 |
| The user interface is clear and understandable                                       | Likert Scale    |
| The system is easy to navigate within the                                            | Likert Scale    |

|                                                                                                                             |              |
|-----------------------------------------------------------------------------------------------------------------------------|--------------|
| standard IntelliJ IDEA workflow                                                                                             |              |
| Refactoring recommendations are presented in an organized, sequenced manner                                                 | Likert Scale |
| The system provides clear explanations of detected code smells.                                                             | Likert Scale |
| Overall interaction with the system is intuitive and developer-friendly                                                     | Likert Scale |
| Security                                                                                                                    |              |
| The system securely handles source code during analysis                                                                     | Likert Scale |
| The system does not expose sensitive project data                                                                           | Likert Scale |
| The use of local ONNX inference models ensures my codebase remains private without needing to send data to external servers | Likert Scale |
| Compatibility                                                                                                               |              |
| The system integrates properly within the IntelliJ environment                                                              | Likert Scale |
| The plugin does not interfere with other installed tools or extensions                                                      | Likert Scale |
| The system's refactoring suggestions seamlessly trigger IntelliJ's native refactoring dialogs                               | Likert Scale |
| Maintainability                                                                                                             |              |
| The system architecture appears modular and structured.                                                                     | Likert Scale |
| The plugin can be updated without affecting its core functionality.                                                         | Likert Scale |
| The recommendations contribute to improving source code maintainability.                                                    | Likert Scale |
| Flexibility                                                                                                                 |              |

|                                                                                             |                 |
|---------------------------------------------------------------------------------------------|-----------------|
| The system can adapt to different project types.                                            | Likert Scale    |
| The system provides sufficient control by allowing configuration of WMCA weights            | Likert Scale    |
| The customizable detection thresholds allow the tool to be tuned to specific project needs. | Likert Scale    |
| Overall Evaluation                                                                          |                 |
| Overall, how satisfied are you with the system?                                             | Likert Scale    |
| Would you use this system in real development work?                                         | Multiple Choice |
| What features did you find most useful?                                                     | Long Text       |
| What improvements would you suggest?                                                        | Long Text       |

**Table 6. Classifier Evaluation Form**

Table 5 shows the survey questionnaire used to assess the usability, functional suitability, and end-user acceptability of the system during the Black Box testing phase, a survey questionnaire will be employed as the primary research methodology.

The survey questionnaire will be used to collect quantitative data from the selected Students respondents after they have tested the IntelliJ plugin on live codebases. The survey will consist of two major parts:

1. User Profile: Gathers basic information regarding the respondent
2. System Evaluation: Consists of specific criteria evaluating the plugin's performance, accuracy of recommendations, and user interface clarity.

To quantify the subjective feedback the survey employs a 5-point Likert Scale. Respondents will indicate their level of agreement with various statements regarding the system, ranging from 5 (Strongly Agree) to 1 (Strongly Disagree). The gathered data will then be subjected to the Weighted Mean statistical treatment to determine the final system acceptability score.

### 3.10.5. Professional Evaluation

To validate the technical reliability and practical applicability of the system, a formal evaluation will be conducted by Information Technology and Computer Science professionals using two separate evaluations for a more comprehensive assessment of the system. The first instrument will focus on the plugin's core static analysis mechanisms, including the machine learning ensemble, prioritization methods, and graph-based sequencing algorithms, in order to assess the accuracy, effectiveness, and technical soundness of the implemented methodologies. The second instrument will evaluate the overall system integration within the IntelliJ IDEA environment, particularly in terms of usability, performance, and practical value in supporting software engineering tasks. Both instruments will also allow the evaluators to provide qualitative feedback, technical comments, and recommendations regarding the overall functionality and effectiveness of the system.

#### 3.10.5.1. Algorithm Evaluation Sheet

The table ## outlines the criteria for evaluating the system's algorithmic architecture:

| Machine Learning & Detection Architecture |             |         |          |
|-------------------------------------------|-------------|---------|----------|
| Algorithm                                 | Description | Remarks | Comments |
| Unified Feature Vector & Normalization    |             |         |          |
| Data Balancing (SMOTE)                    |             |         |          |
| Ensemble Architecture                     |             |         |          |
| Per-Tool Smell Heuristics                 |             |         |          |
| Gating Network & Sparse Activation        |             |         |          |

|                                                         |  |  |  |
|---------------------------------------------------------|--|--|--|
| (TopK)                                                  |  |  |  |
| Expert Accuracy Matrix & Final Inference                |  |  |  |
| Prioritization & Sequencing Logic                       |  |  |  |
| Refactoring Impact Simulation (AST Deltas)              |  |  |  |
| WMCA Prioritization Formula                             |  |  |  |
| DCAS Tiers 1 & 2 (Priority & Semantic Sorting)          |  |  |  |
| DCAS Tiers 3 & 4 (Conflict Resolution & Cycle-Breaking) |  |  |  |
| Overall Viability & Actionability                       |  |  |  |
| Computational Scalability                               |  |  |  |
| Practical Actionability & Priority Tiers                |  |  |  |
| General Feedback                                        |  |  |  |
|                                                         |  |  |  |

### 3.10.5.2. System Integration Evaluation Sheet

The table ## outlines the criteria for evaluating the practical usage and implementation of the IntelliJ plugin:

| 1. IDE Integration & User Experience |                                                                                                                              |         |          |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------|---------|----------|
| Category                             | Description                                                                                                                  | Remarks | Comments |
| Native IntelliJ Integration          | Evaluates how seamlessly the plugin embeds into the IntelliJ IDEA ecosystem without disrupting standard developer workflows. |         |          |

|                                |                                                                                                                                                                                                           |  |  |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
| Information Architecture       | Evaluates the clarity of the UI, specifically how well the tool presents the High (Critical), Medium (Moderate), and Low priority refactoring tiers to the user without visual clutter.                   |  |  |
| 2. Performance & Stability     |                                                                                                                                                                                                           |  |  |
| IDE Responsiveness             | Evaluates the execution speed of the underlying static analysis logic and heuristics. Ensures that the local ONNX inference, AST extractions, and graph sorting do not cause the IDE UI to freeze or lag. |  |  |
| Resource Management            | Evaluates the plugin's CPU and memory footprint during active codebase scanning and virtual refactoring simulations.                                                                                      |  |  |
| 3. Practical Developer Utility |                                                                                                                                                                                                           |  |  |
| Actionability of Roadmap       | Evaluates if the step-by-step refactoring sequence provided by the tool is logical, safe to execute, and practically useful for a developer trying to resolve technical debt.                             |  |  |
| Explainability & Transparency  | Evaluates how well the plugin communicates why a specific refactoring is recommended (e.g.,                                                                                                               |  |  |



|                                       |                                                                                                                                                                                                     |  |  |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
|                                       | surfacing the structural ROI, security risk, or showing why a certain sequence was chosen to break a cycle).                                                                                        |  |  |
| 4. General Software Engineering Value |                                                                                                                                                                                                     |  |  |
| Code Quality Impact                   | Evaluates the professional's assessment of whether following the plugin's recommendations would realistically improve a project's maintainability, cohesion, and coupling in a real-world scenario. |  |  |
| General Feedback                      |                                                                                                                                                                                                     |  |  |
|                                       |                                                                                                                                                                                                     |  |  |

### 3.11. Respondents of the Study and Sampling Technique

The study will use Human Evaluators Method during Black Box Testing to evaluate three aspects of IntelliJ plugin performance which include its functional capabilities and system reliability and user interface design. The method enables real users to test the plugin while they deliver structured assessments of its performance and user experience and its success at identifying code smells and refactoring those issues.

#### 1. Sampling Technique

The research employs Purposive Sampling which selects respondents based on their technical expertise and programming skills and their knowledge of software development

tools. The study requires this sampling method because the IntelliJ plugin needs to be tested among software engineers who use this specialized tool for their work.

The research needs to exclude random sampling because people who lack programming skills and Integrated Development Environment (IDE) knowledge cannot properly test the plugin or assess its refactoring recommendations.

## **2. Target Population and Inclusion Criteria**

The study selects its target population from students who study Computer Science and Information Technology and from IT professionals and CS professionals who have Java development experience. The evaluation results require all respondents to meet the following specific criteria which will establish both reliability and validity:

- **Technical Proficiency**

The candidate should have basic or in-depth knowledge of the Java programming language.

- **Environment Familiarity**

The candidate should have prior experience working with Integrated Development Environments (IDEs), particularly IntelliJ IDEA to ensure they can easily navigate the plugin's UI.

- **Domain Knowledge**

The candidate should have basic knowledge of object-oriented programming concepts, software maintenance, and basic code structure.

## **3. Sample Size**

The study will require fifty (50) respondents to achieve a statistically reliable Weighted Mean which will assess system performance. The selected sample size enables a fair

assessment of the system through its combination of expert evaluations and student usability testing.

The IT and CS experts will conduct an assessment of the refactoring recommendations which includes determining their technical accuracy and practical applicability and overall validity. The IT and CS students will evaluate the plugin through its user interface which includes testing its usability and learning process and complete user experience.

| <b>Respondent Category</b> | <b>Number of Respondents</b> |
|----------------------------|------------------------------|
| IT Professionals           | 3                            |
| CS Professionals           | 2                            |
| IT/CS Students             | 45                           |
| Total                      | 50                           |

**Table 7. Distribution of Respondents**

The students from different academic programs show different priorities when they assess their evaluation criteria which their academic programs require. Computer Science students give more importance to system processes because both groups want accessible systems that require less maintenance work, while Information Technology students concentrate on system operation and user interface design.

The professional group assesses systems by using technical standards which emphasize the evaluation of software maintenance through three criteria: refactoring validity, structural correctness, and software maintenance usability. The experts assess recommendations based on

two factors: their capacity to affect system architecture and their compliance with industrial standards.

### 3.12. Statistical Treatment

The data collected during the testing and evaluation phases of the study will undergo appropriate statistical analysis to interpret the results of the machine learning model as well as the acceptance of the system by the end-user.

#### 3.12.1. Descriptive Statistics

Descriptive statistics in the form of Mean ( $\mu$ ) and Standard Deviation ( $\sigma$ ) is used to summarize the performance of the system in the various software projects in the corpus.

- Mean: Is used to find the average Execution Time (ms) and Memory Consumption (MB) of the plugin.
- Standard Deviation: Helps in measuring the variability and stability of the system's performance, while undertaking projects of different sizes.

#### 3.12.2. Usability and System Acceptance (Software Evaluation)

To quantify the qualitative feedback gathered from the Black Box testing phase (End-User Evaluation), the Likert Scale and Weighted Mean will be employed.

##### 3.12.2.1. The Likert Scale

| Likert Scale      |         |         |       |                |
|-------------------|---------|---------|-------|----------------|
| 1                 | 2       | 3       | 4     | 5              |
| Strongly Disagree | Neutral | Neutral | Agree | Strongly Agree |
| Weight            |         |         |       |                |
| 1                 | 2       | 3       | 4     | 5              |

### **Table 8. Likert Scale Weighting**

Table 7 presents a 5-point Likert Scale will be used for measuring the level of respondents' agreement regarding the functional suitability, reliability, and usability of the system. The numerical weights are assigned as follows:

#### **3.12.2.2. Weighted Mean**

The Weighted Mean will be computed to ascertain the average response for each evaluation category. This statistical equation will sum up the frequency of responses to ascertain the consensus of all human testers.

$$WM = \frac{\sum fx}{N}$$

#### **Equation 19. Weighted Mean**

Where:

- WM = Weighted Mean
- f = Frequency of responses for a specific scale
- x = The numerical weight of the scale (1 to 5)
- $\sum fx$  = The sum of the products of the frequency and the weight
- N = Total number of respondents

### 3.12.3. Comparative Analysis

To determine whether the proposed Random Forest MoE-inspired confidence estimation and Weighted Multi-Criteria Analysis (WMCA) improve recommendation quality, the system will be compared against baseline methods using classification, ranking, and before-and-after structural metrics.

For confidence estimation, the Random Forest model will be evaluated using Precision, Recall, F1-score, and Brier Score, and compared with simpler baselines such as rule-only detection, Decision Tree, or Logistic Regression where applicable.

For prioritization, the WMCA ranking will be compared with severity-only ranking, confidence-only ranking, rule-only ranking, and random ranking. Precision@K and NDCG@K will be used to evaluate whether the proposed approach ranks more relevant recommendations higher than the baselines.

For before-and-after evaluation, the same number of top-ranked recommendations from the proposed approach and each baseline will be applied or evaluated. Structural metric changes will be measured using LOC, Cyclomatic Complexity, CBO, LCOM, and WMC:

$$\Delta\text{Metric} = \text{Metric\_before} - \text{Metric\_after}$$

## **Equation 20. $\Delta$ Metric**

Mean and standard deviation will summarize the metric changes. When applicable, a paired t-test or Wilcoxon signed-rank test will be used to determine whether the before-and-after differences are statistically meaningful.

The results will be interpreted within the study's scope and will not be treated as proof of universal maintainability improvement or behavioral correctness.

Objective algorithmic evaluation is incorporated to reduce dependence on subjective human judgment. Even if novice respondents are unable to fully assess maintainability quality, the effectiveness of recommendations can still be measured through structural metric improvements, ranking quality metrics, and comparison against baseline prioritization methods. This ensures that recommendation quality is evaluated using both human-centered and quantitative software engineering measures.

## **References**

- [1] Mens, T., & Tourwe, T. (2004). A survey of software refactoring. *IEEE Transactions on Software Engineering*, 30(2), 126–139. <https://doi.org/10.1109/tse.2004.1265817>

- [2] Baqais, A. A. B., & Alshayeb, M. (2019). Automatic software refactoring: a systematic literature review. *Software Quality Journal*. <https://doi.org/10.1007/s11219-019-09477-y>
- [3] AlOmar, E. A., Mkaouer, M. W., & Ouni, A. (2023). Behind the Intent of Extract Method Refactoring: A Systematic Literature Review. *ArXiv.org*. <https://arxiv.org/abs/2312.12600>
- [4] Almogahed, A., Omar, M., Zakaria, N. H., Muhammad, G., & AlQahtani, S. A. (2022). Revisiting Scenarios of Using Refactoring Techniques to Improve Software Systems Quality. *IEEE Access*, 1–1. <https://doi.org/10.1109/access.2022.3218007>
- [5] Hodovychenko, M. A., & Kurinko, D. D. (2025). Analysis of existing approaches to automated refactoring of object-oriented software systems. *Herald of Advanced Information Technology*, 8(2), 179–196. <https://doi.org/10.15276/hait.08.2025.11>
- [6] Akhtar, S. M., Nazir, M., Ali, A., Khan, A. S., Atif, M., & Naseer, M. (2022). A Systematic Literature Review on Software- refactoring Techniques, Challenges, and Practices. *VFAST Transactions on Software Engineering*, 10(4), 93–103. <https://doi.org/10.21015/vtse.v10i4.1275>
- [7] Recupito, G., Giordano, G., Ferrucci, F., Di Nucci, D., & Palomba, F. (2025). When code smells meet ML: on the lifecycle of ML-specific code smells in ML-enabled systems. *Empirical Software Engineering*, 30(5). <https://doi.org/10.1007/s10664-025-10676-4>
- [8] Kaur, A. (2019). A Systematic Literature Review on Empirical Analysis of the Relationship Between Code Smells and Software Quality Attributes. *Archives of Computational Methods in Engineering*. <https://doi.org/10.1007/s11831-019-09348-6>
- [9] Kaur, A. (2019). A Systematic Literature Review on Empirical Analysis of the Relationship Between Code Smells and Software Quality Attributes. *Archives of Computational Methods in Engineering*. <https://doi.org/10.1007/s11831-019-09348-6>



- [10] Engeln, U. (2024). Code Smell Detection using Features from Version History. *DL.gi.de*, 173–174. [https://doi.org/10.18420/sw2024-ws\\_13](https://doi.org/10.18420/sw2024-ws_13)
- [11] Brito, A., Hora, A., & Valente, M. T. (2020). Refactoring Graphs: Assessing Refactoring over Time. <https://doi.org/10.1109/saner48275.2020.9054864>
- [12] Pereira, G. P., Francisco, L., & Iorio, V. D. (2025). Building Refactoring Tools: Insights from RefactorEx. *Workshop Em Engenharia de Software Para Programação Funcional*, 9–14. <https://doi.org/10.5753/se4fp.2025.13088>
- [13] Kaur, A. (2020). A Systematic Literature Review on Empirical Analysis of the Relationship Between Code Smells and Software Quality Attributes: A. Kaur. *Archives of Computational Methods in Engineering*, 27(4), 1267-1296.
- [14] Alfadel, M., Aljasser, K., & Alshayeb, M. (2020). Empirical study of the relationship between design patterns and code smells. *Plos one*, 15(4), e0231731.
- [15] Anquetil, N., Etien, A., Andreo, G., & Ducasse, S. (2019, September). Decomposing god classes at siemens. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)* (pp. 169-180). IEEE.
- [16] Digkas, G. (2021). Reducing Technical Debt Density: Refactoring vs. Writing Clean New Code. *Data Archiving and Networked Services (DANS)*. <https://doi.org/10.33612/diss.181245089>
- [17] Muhammad Fawad, Ghulam Rasoo, & Riaz, M. B. (2025). Refactoring Android Source Code Smells from Android Applications. *IEEE Access*, 1–1. <https://doi.org/10.1109/access.2025.3529687>
- [18] Silva, D. R., Santana, Santos, G., Ricardo Mingarini Terra, & Marco Tulio Valente. (2020). RefDiff 2.0: A Multi-Language Refactoring Detection Tool. 47(12), 2786–2802. <https://doi.org/10.1109/tse.2020.2968072>

- [19] Tsantalis, N., & Chatzigeorgiou, A. (2010). Identification of refactoring opportunities introducing polymorphism. *Journal of Systems and Software*, 83(3), 391–404. <https://doi.org/10.1016/j.jss.2009.09.017>
- [20] Mohan, M., & Greer, D. (2018). Automated Refactoring of Software using Version History and a Code Element Recentness Measure. *Proceedings of the 13th International Conference on Evaluation of Novel Approaches to Software Engineering*, 455–462. <https://doi.org/10.5220/0006815304550462>
- [21] Nursapa, S., Samuilova, A., Bucaioni, A., & Nguyen, P. T. (2025). ROSE: Transformer-Based Refactoring Recommendation for Architectural Smells. *ArXiv.org*. <https://arxiv.org/abs/2507.12561v2>
- [22] Gupta, R., & Kumar Singh, S. (2021). A Novel Metric based Detection of Temporary Field Code Smell and its Empirical Analysis. *Journal of King Saud University - Computer and Information Sciences*. <https://doi.org/10.1016/j.jksuci.2021.11.005>
- [23] Almogahed, A., Omar, M., & Zakaria, N. H. (2022). Refactoring Codes to Improve Software Security Requirements. *Procedia Computer Science*, 204, 108–115. <https://doi.org/10.1016/j.procs.2022.08.013>
- [24] De Stefano, M., Gambardella, M., Pecorelli, F., Palomba, F., & De Lucia, A. (n.d.). cASpER: A Plug-in for Automated Code Smell Detection and Refactoring. <https://doi.org/10.1145/3399715.3399955>
- [25] Maini, R., Kaur, N., & Kaur, A. (2024). HSHEP: An Optimization-Based Code Smell Refactoring Sequencing Technique. *Journal of Computational and Cognitive Engineering*. <https://doi.org/10.47852/bonviewjccce42023180>

- [26] Wu, D., Mu, F., Shi, L., Guo, Z., Liu, K., Zhuang, W., Zhong, Y., & Zhang, L. (2024). iSMELL: Assembling LLMs with Expert Toolsets for Code Smell Detection and Refactoring. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 1345–1357. <https://doi.org/10.1145/3691620.3695508>
- [27] Moghadam, I. H., Cinneide, M. O., Zarepour, F., & Jahanmir, M. A. (2021). RefDetect: A Multi-Language Refactoring Detection Tool based on String Alignment. *IEEE Access*, 1–1. <https://doi.org/10.1109/access.2021.3086689>
- [28] Maini, R., Kaur, N., & Kaur, A. (2025). Optimized Refactoring Sequence for Object-Oriented Code Smells. *International Journal of Environmental Sciences*, 11(7s), 593–612. <https://doi.org/10.64252/6r5jn666>
- [29] Pourasghar, B., Izadkhah, H., Isazadeh, A., & Lotfi, S. (2021). A graph-based clustering algorithm for software systems modularization. *Information and Software Technology*, 133, 106469. <https://doi.org/10.1016/j.infsof.2020.106469>
- [30] Hou, D., Yin, Y., Su, Q., & Liu, L. (2020). Software Refactoring Scheme Based on NSGA-II Algorithm. 2006, 447–452. <https://doi.org/10.1109/dsa51864.2020.00076>
- [31] Robredo, Mikel & Esposito, Matteo & Palomba, Fabio & Peñaloza, Rafael & Lenarduzzi, Valentina. (2024). Analyzing the Ripple Effects of Refactoring. A Registered Report. [https://www.researchgate.net/publication/384056632\\_Analyzing\\_the\\_Ripple\\_Effects\\_of\\_Refactoring\\_A\\_Registered\\_Report](https://www.researchgate.net/publication/384056632_Analyzing_the_Ripple_Effects_of_Refactoring_A_Registered_Report)
- [32] Tom, E., Aurum, A., & Vidgen, R. (2013). An exploration of technical debt. *Journal of Systems and Software*, 86(6), 1498–1516. <https://doi.org/10.1016/j.jss.2012.12.052>

- [33] P. Martins, R. Achar and C. V. Lopes, "50K-C: A Dataset of Compilable, and Compiled, Java Projects," 2018 IEEE/ACM 15th International Conference on Mining Software Repositories (MSR). <https://ieeexplore.ieee.org/document/8595165>
- [34] Li, M., & Fazzini, M. (2025). DataTD: A Dataset of Java Projects Including Test Doubles. 851–855. <https://doi.org/10.1109/msr66628.2025.00124>
- [35] Kalfin Kalfin, Ibrahim, R. A., & Grida Sakti Laksito. (2024). Optimization of White Box Testing by Utilizing Branching and Repeating Structures in Java Programs Using Base Path. International Journal of Mathematics Statistics and Computing, 2(2), 85–89. <https://doi.org/10.46336/ijmsc.v2i2.98>
- [36] Khan, Y. A., Elish, M. O., & El-Attar, M. (2012). A Systematic Review on the Impact of CK Metrics on the Functional Correctness of Object-Oriented Classes. Lecture Notes in Computer Science, 258–273. [https://doi.org/10.1007/978-3-642-31128-4\\_19](https://doi.org/10.1007/978-3-642-31128-4_19)
- [37] Sujon, K. M., Hassan, R., Choi, K., & Samad, M. A. (2025). Accuracy, precision, recall, fl-score, or MCC? empirical evidence from advanced statistics, ML, and XAI for evaluating business predictive models. Journal of Big Data, 12(1). <https://doi.org/10.1186/s40537-025-01313-4>
- [38] Palomba, F., Bavota, G., Penta, M. D., Oliveto, R., Poshyvanyk, D., & De Lucia, A. (2015). Mining Version Histories for Detecting Code Smells. IEEE Transactions on Software Engineering, 41(5), 462–489. <https://doi.org/10.1109/tse.2014.2372760>
- [39] Lech Madeyski, & Tomasz Lewowski. (2020). MLCQ: Industry-Relevant Code Smell Data Set. <https://doi.org/10.1145/3383219.3383264>
- [40] Install IntelliJ IDEA - Help | IntelliJ IDEA. (n.d.). [Www.jetbrains.com. https://www.jetbrains.com/help/idea/installation-guide.html](https://www.jetbrains.com/help/idea/installation-guide.html)

- [41] Sujon, K. M., Hassan, R., Choi, K., & Samad, M. A. (2025). Accuracy, precision, recall, f1-score, or MCC? empirical evidence from advanced statistics, ML, and XAI for evaluating business predictive models. *Journal of Big Data*, 12(1). <https://doi.org/10.1186/s40537-025-01313-4>
- [42] Marios Fokaefs, Nikolaos Tsantalis, Stroulia, E., & Chatzigeorgiou, A. (2012). Identification and application of Extract Class refactorings in object-oriented systems. 85(10), 2241–2260. <https://doi.org/10.1016/j.jss.2012.04.013>
- [43] Nikolaos Tsantalis, & Chatzigeorgiou, A. (2011). Identification of extract method refactoring opportunities for the decomposition of methods. *Journal of Systems and Software*, 84(10), 1757–1782. <https://doi.org/10.1016/j.jss.2011.05.016>
- [44] Tsantalis, N., & Chatzigeorgiou, A. (2009). Identification of Move Method Refactoring Opportunities. *IEEE Transactions on Software Engineering*, 35(3), 347–367. <https://doi.org/10.1109/tse.2009.1>
- [45] Tsantalis, N., & Chatzigeorgiou, A. (2010). Identification of refactoring opportunities introducing polymorphism. *Journal of Systems and Software*, 83(3), 391–404. <https://doi.org/10.1016/j.jss.2009.09.017>
- [46] Vidal, S. A., Marcos, C., & Díaz-Pace, J. A. (2014). An approach to prioritize code smells for refactoring. *Automated Software Engineering*, 23(3), 501–532. <https://doi.org/10.1007/s10515-014-0175-x>
- [47] Huang, Z., Yu, H., Fan, G., Shao, Z., Zhou, Z., & Li, M. (2024). On the effectiveness of developer features in code smell prioritization: A replication study. *Journal of Systems and Software*, 210, 111968. <https://doi.org/10.1016/j.jss.2024.111968>

- [48] Pecorelli, F., Palomba, F., Khomh, F., & De Lucia, A. (2020). Developer-Driven Code Smell Prioritization. Proceedings of the 17th International Conference on Mining Software Repositories. <https://doi.org/10.1145/3379597.3387457>
- [49] Avnimelech, R., & Intrator, N. (1999). Boosted Mixture of Experts: An Ensemble Learning Scheme. *Neural Computation*, 11(2), 483–497. <https://doi.org/10.1162/089976699300016737>
- [50] Gan, W., Ning, Z., Qi, Z., & Yu, P. S. (2025). Mixture of experts (MoE): A big data perspective. *Information Fusion*, 103664. <https://doi.org/10.1016/j.inffus.2025.103664>
- [51] Islam, M. R., Al Maruf, A., & Cerny, T. (2022). Code Smell Prioritization with Business Process Mining and Static Code Analysis: A Case Study. *Electronics*, 11(12), 1880. <https://doi.org/10.3390/electronics11121880>
- [52] Kokol, P., Kokol, M., & Zagoranski, S. (2021). Code smells: A Synthetic Narrative Review. *ArXiv.org*. <https://arxiv.org/abs/2103.01088>
- [53] Palomba, F., Bavota, G., Di Penta, M., Fasano, F., Oliveto, R., & De Lucia, A. (2018). On the diffuseness and the impact on maintainability of code smells. Proceedings of the 40th International Conference on Software Engineering, 482–482. <https://doi.org/10.1145/3180155.3182532>
- [54] Zhang, Z., Wen, E., & Tempero, E. (2025). Analyzing Variations in Dependency Distributions Due to Code Smell Interactions. *ArXiv.org*. <https://arxiv.org/abs/2509.03896>
- [55] Sehgal, R., Mehrotra, D., Nagpal, R., & Sharma, R. (2020). Green software: Refactoring approach. *Journal of King Saud University - Computer and Information Sciences*. <https://doi.org/10.1016/j.jksuci.2020.10.022>
- [56] Technical Debt: From Metaphor to Theory and Practice. (n.d.). *Ieeexplore.ieee.org*. <https://ieeexplore.ieee.org/document/6336722>

- [57] Yamashita, A., & Moonen, L. (n.d.). Do code smells reflect important maintainability aspects? Retrieved February 12, 2026, from <https://web-backend.simula.no/sites/default/files/publications/Simula.simula.1278.pdf>
- [58] Yli-Huumo, J., Maglyas, A., & Smolander, K. (2016). The Effects of Software Process Evolution to Technical Debt—Perceptions from Three Large Software Projects. *Managing Software Process Evolution*, 305–327. [https://doi.org/10.1007/978-3-319-31545-4\\_15](https://doi.org/10.1007/978-3-319-31545-4_15)
- [59] Palomba, F., Bavota, G., Di Penta, M., Fasano, F., Oliveto, R., & De Lucia, A. (2018). On the diffuseness and the impact on maintainability of code smells. *Proceedings of the 40th International Conference on Software Engineering*, 482–482. <https://doi.org/10.1145/3180155.3182532>
- [60] Fucci, D., Romano, S., Baldassarre, M., Caivano, D., Scanniello, G., Thuran, B., & Juristo, N. (2022). A Longitudinal Cohort Study on the Retainment of Test-Driven Development. *ArXiv*, 1(1). <https://arxiv.org/pdf/1807.02971.pdf>
- [61] Kokol, P., Kokol, M., & Zagoranski, S. (2021). Code smells: A Synthetic Narrative Review. *ArXiv.org*. <https://arxiv.org/abs/2103.01088>
- [62] Palomba, F., Bavota, G., Di Penta, M., Fasano, F., Oliveto, R., & De Lucia, A. (2018). On the diffuseness and the impact on maintainability of code smells. *Proceedings of the 40th International Conference on Software Engineering*, 482–482. <https://doi.org/10.1145/3180155.3182532>
- [63] Almogahed, A., Omar, M., Zakaria, N. H., Muhammad, G., & AlQahtani, S. A. (2022). Revisiting Scenarios of Using Refactoring Techniques to Improve Software Systems Quality. *IEEE Access*. <https://doi.org/10.1109/access.2022.3218007>

- [64] Almogahed, A., Omar, M., & Zakaria, N. H. (2022). Refactoring Codes to Improve Software Security Requirements. *Procedia Computer Science*, 204, 108–115. <https://doi.org/10.1016/j.procs.2022.08.013>
- [65] Zazworka, N., Vetro', A., Izurieta, C., Wong, S., Cai, Y., Seaman, C., & Shull, F. (2013). *Software Quality Journal*, 22(3), 403–426. doi:10.1007/s11219-013-9200-8
- [66] Anquetil, N., & Lethbridge, T. C. (2019). An approach of extracting God class exploiting both structural and semantic similarity. *Proceedings of the 14th International Conference on Software Technologies*, 370–379. <https://www.scitepress.org/Papers/2019/77438/77438.pdf>
- [67] Zazworka, N., Shaw, M. A., Shull, F., & Seaman, C. (2011, May). Investigating the impact of design debt on software quality. In *Proceedings of the 2nd workshop on managing technical debt* (pp. 17-23).
- [68] Fokaefs, M., Tsantalis, N., & Chatzigeorgiou, A. (2007). JDeodorant: Identification and removal of Feature Envy bad smells. *Proceedings of the 23rd IEEE International Conference on Software Maintenance (ICSM)*, 47–55. <https://doi.org/10.1109/ICSM.2007.4362679>
- [69] Liu, Y., Zhang, Y., & others. (2022). Correlation feature mining model based on dual information fusion for feature envy detection. *Proceedings of the 34th International Conference on Software Engineering & Knowledge Engineering (SEKE)*. <https://ksiresearch.org/seke/seke22paper/paper009.pdf>
- [70] Wang, J., Xiao, X., & Zhang, H. (2021). AdaBoost-based Refused Bequest code smell detection with synthetic smelly instances. *2021 IEEE 29th International Conference on Program Comprehension (ICPC)*, 255–265. <https://ieeexplore.ieee.org/document/9331242>



- [71] Palomba, F., Bavota, G., Di Penta, M., Fasano, F., Oliveto, R., & De Lucia, A. (2018). On the diffuseness and the impact on maintainability of code smells. *Empirical Software Engineering*, 23, 1188-1221. <https://doi.org/10.1007/s10664-017-9535-z>
- [72] Yamashita, A., & Moonen, L. (2012). Do code smells reflect important maintainability aspects? 28th IEEE International Conference on Software Maintenance, 306-315. <https://doi.org/10.1109/ICSM.2012.6405287>